Computer Architecture

Control Unit*

Amir Mahdi Hosseini Monazzah

Room 332,
School of Computer Engineering,
Iran University of Science and Technology,
Tehran, Iran.

monazzah@iust.ac.ir

Spring 2025

*This chapter is based on the basic computer design by Morris Mano. Parts of slides are from Computer System Architecture by guiseok@yahoo.com (no further information)

Outline

- Introduction
  - What is control unit?
  - Preliminaries
- Hardwire control
  - Essential modules
  - Instruction cycle
  - Control unit design
- Microprogrammed control

Control unit

- Control unit initiate sequences of microoperations
  - Control signal *(that specify microoperations)* in a bus-organized system
    - Groups of bits that select the paths in multiplexers, decoders, and arithmetic logic units
- Two major types of control unit
  - Hardwired control
    - The control logic is implemented with gates, F/Fs, decoders, and other digital circuits
    - + Fast operation, - Wiring change (if the design has to be modified)

Control unit

- Control unit initiate sequences of microoperations
  - Control signal *(that specify microoperations)* in a bus-organized system
    - Groups of bits that select the paths in multiplexers, decoders, and arithmetic logic units
- Two major types of control unit
  - Microprogrammed control
    - The control information is stored in a control memory, and the control memory is programmed to initiate the required sequence of microoperations
    - + Any required change can be done by updating the microprogram in control memory, - Slow operation

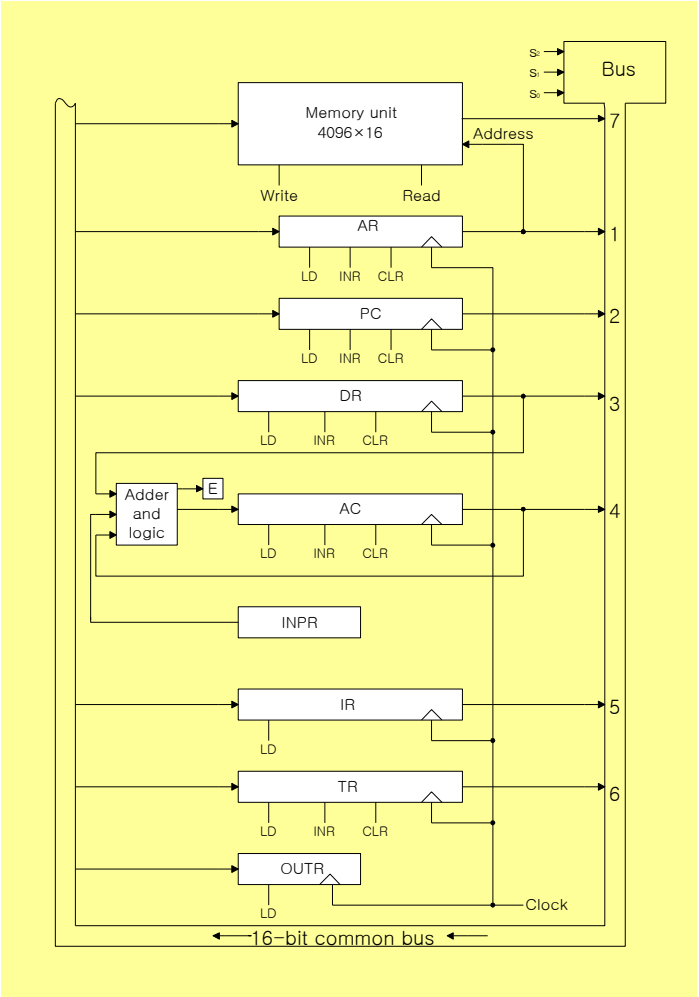
Timing and control

- Clock pulses
  - A master clock generator controls the timing for all registers in the computer
  - The clock pulses are applied to all F/Fs and registers in system
  - The clock pulses do not change the state of a register unless the register is enabled by a control signal
  - The control signals are generated in the control unit
    - The control signals provide control inputs for the multiplexers in the common bus, control inputs in processor registers, microoperations for the ALU and etc.

Mano computer!

- Basic computer architecture

  by Morris Mano



Timing and control

- Control unit
  - Control unit = Control logic gate + 3X8 decoder + instruction register + timing signal
  - Timing signal = 4X16 Decoder + 4-bit Sequence Counter

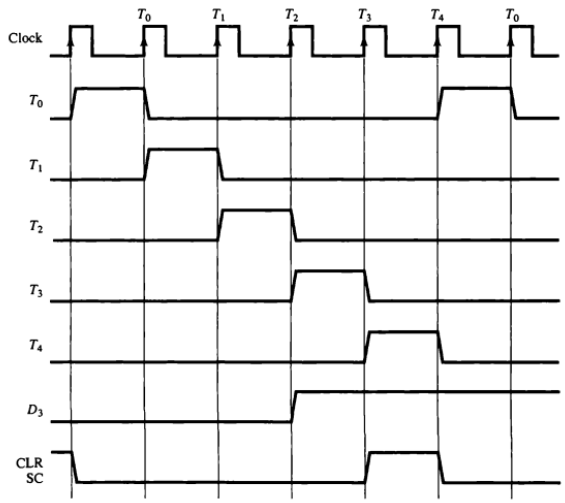
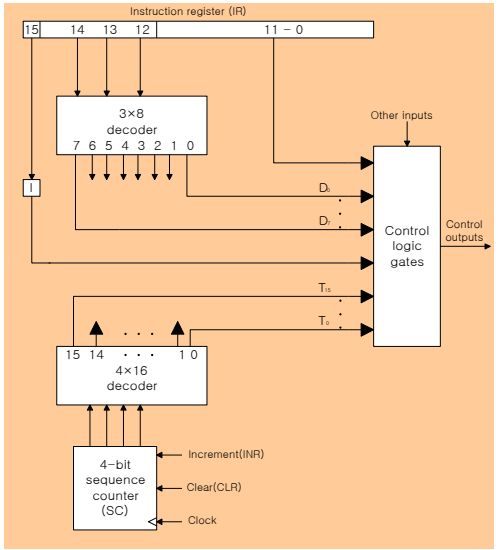


Figure 5-7 Example of control timing signals.



Timing and control

- Control unit
  - Example) Control timing
    - Sequence counter is cleared when $D_3T_4 = 1$: $D_3T_4 : SC \leftarrow 0$

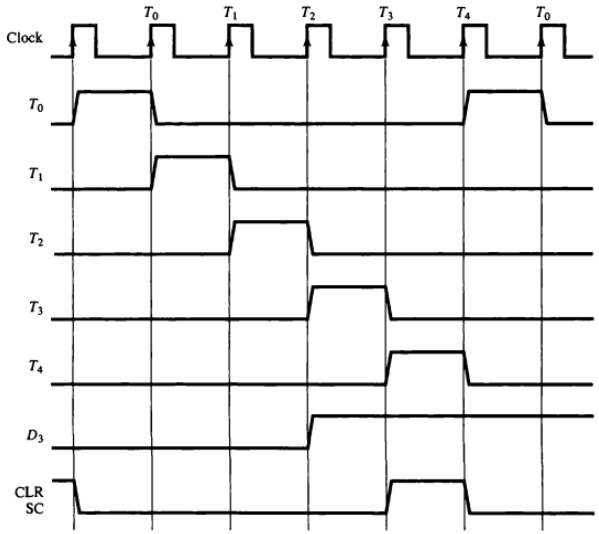
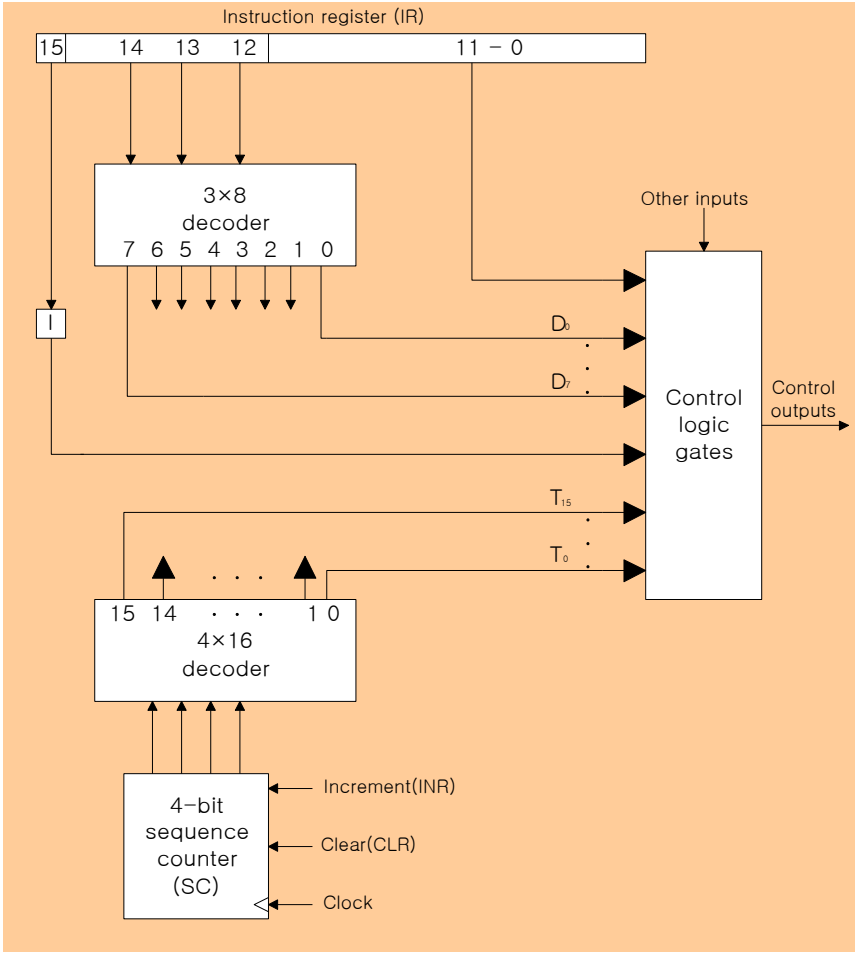


Figure 5-7 Example of control timing signals.



Timing and control

- Example) Register transfer statement : $T_0 : AR \leftarrow PC$
  - A transfer of the content of PC into AR if timing signal T_0 is active
    - 1) During T_0 active, the content of PC is placed onto the bus $(S_2 S_1 S_0)$
    - 2) LD (load) input of AR is enabled, the actual transfer occurs at the next positive transition of the clock (T_1 rising edge clock)
    - 3) SC (sequence counter) is incremented

$0000(T_0) \rightarrow 0001(T_1)$

T_0 : Inactive
T_1 : Active

Designing control logic

- Register control example: AR
  - Control inputs of AR:
    - ***LD, INR, CLR***
  - Find all the statements that change the AR

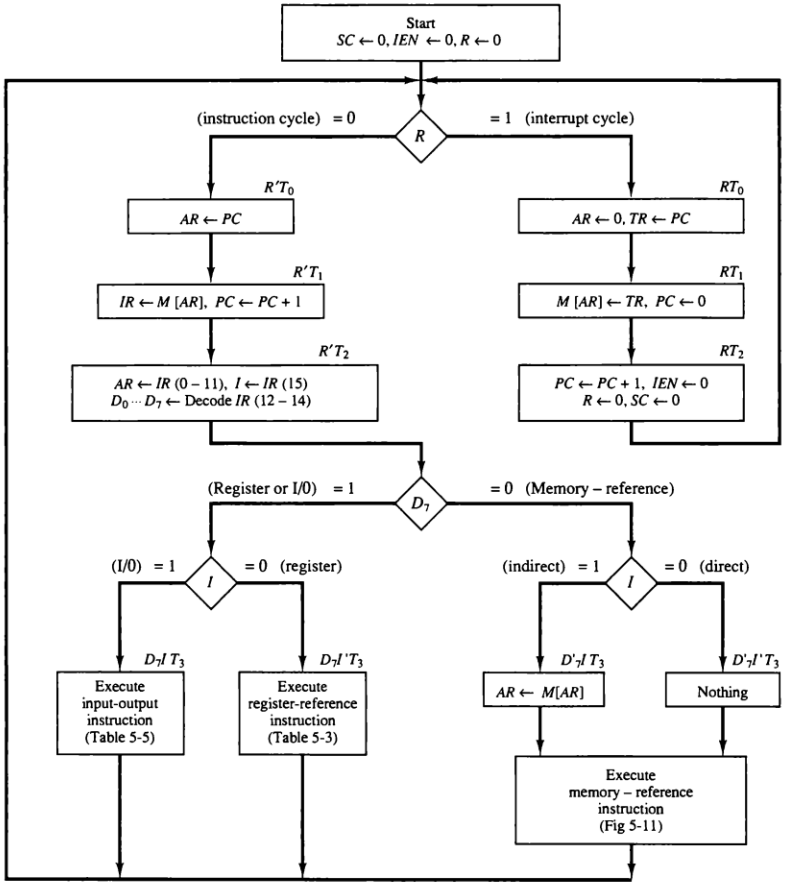


Figure 5-15 Flowchart for computer operation.

Designing control logic

- Register control example: AR
  - Control inputs of AR:
    - ***LD, INR, CLR***
  - Find all the statements that change the AR

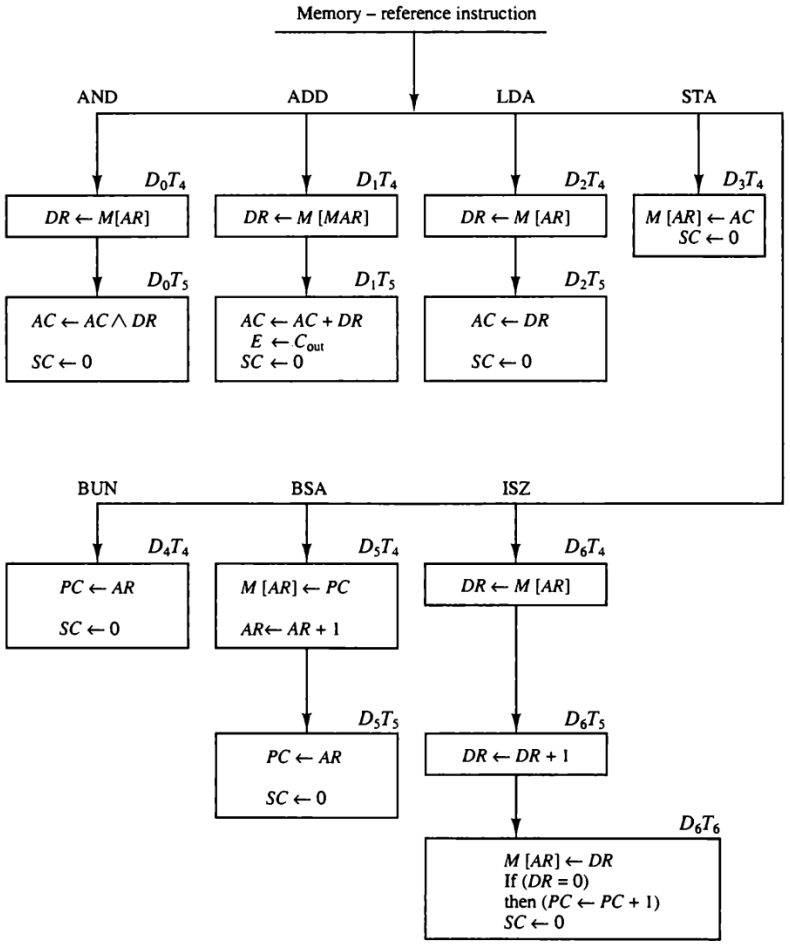


Figure 5-11 Flowchart for memory-reference instructions.

Designing control logic

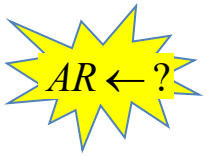
- Register control example: AR
  - Control inputs of AR:
    - ***LD, INR, CLR***
  - Find all the statements that change the AR

TABLE 5-6 Control Functions and Microoperations for the Basic Computer

Fetch	$R'T_0$:	$AR \leftarrow PC$
	$R'T_1$:	$IR \leftarrow M[AR]$, $PC \leftarrow PC + 1$
Decode	$R'T_2$:	$D_0, \ldots, D_7 \leftarrow$ Decode $IR(12\text{–}14)$, $AR \leftarrow IR(0\text{–}11)$, $I \leftarrow IR(15)$
Indirect	$D_7'IT_3$:	$AR \leftarrow M[AR]$
Interrupt:		
$T_0'T_1'T_2'(IEN)(FGI + FGO)$:		$R \leftarrow 1$
	RT_0:	$AR \leftarrow 0$, $TR \leftarrow PC$
	RT_1:	$M[AR] \leftarrow TR$, $PC \leftarrow 0$
	RT_2:	$PC \leftarrow PC + 1$, $IEN \leftarrow 0$, $R \leftarrow 0$, $SC \leftarrow 0$
Memory-reference:		
AND	D_0T_4:	$DR \leftarrow M[AR]$
	D_0T_5:	$AC \leftarrow AC \wedge DR$, $SC \leftarrow 0$
ADD	D_1T_4:	$DR \leftarrow M[AR]$
	D_1T_5:	$AC \leftarrow AC + DR$, $E \leftarrow C_{out}$, $SC \leftarrow 0$
LDA	D_2T_4:	$DR \leftarrow M[AR]$
	D_2T_5:	$AC \leftarrow DR$, $SC \leftarrow 0$
STA	D_3T_4:	$M[AR] \leftarrow AC$, $SC \leftarrow 0$
BUN	D_4T_4:	$PC \leftarrow AR$, $SC \leftarrow 0$
BSA	D_5T_4:	$M[AR] \leftarrow PC$, $AR \leftarrow AR + 1$
	D_5T_5:	$PC \leftarrow AR$, $SC \leftarrow 0$
ISZ	D_6T_4:	$DR \leftarrow M[AR]$
	D_6T_5:	$DR \leftarrow DR + 1$
	D_6T_6:	$M[AR] \leftarrow DR$, if $(DR = 0)$ then $(PC \leftarrow PC + 1)$, $SC \leftarrow 0$
Register-reference:		
		$D_7I'T_3 = r$ (common to all register-reference instructions)
		$IR(i) = B_i$ $(i = 0, 1, 2, \ldots, 11)$
	r:	$SC \leftarrow 0$
CLA	rB_{11}:	$AC \leftarrow 0$
CLE	rB_{10}:	$E \leftarrow 0$
CMA	rB_9:	$AC \leftarrow \overline{AC}$
CME	rB_8:	$E \leftarrow \overline{E}$
CIR	rB_7:	$AC \leftarrow$ shr AC, $AC(15) \leftarrow E$, $E \leftarrow AC(0)$
CIL	rB_6:	$AC \leftarrow$ shl AC, $AC(0) \leftarrow E$, $E \leftarrow AC(15)$
INC	rB_5:	$AC \leftarrow AC + 1$
SPA	rB_4:	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$
SNA	rB_3:	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$
SZA	rB_2:	If $(AC = 0)$ then $PC \leftarrow PC + 1$
SZE	rB_1:	If $(E = 0)$ then $(PC \leftarrow PC + 1)$
HLT	rB_0:	$S \leftarrow 0$
Input-output:		
		$D_7IT_3 = p$ (common to all input–output instructions)
		$IR(i) = B_i$ $(i = 6, 7, 8, 9, 10, 11)$
	p:	$SC \leftarrow 0$
INP	pB_{11}:	$AC(0\text{–}7) \leftarrow INPR$, $FGI \leftarrow 0$
OUT	pB_{10}:	$OUTR \leftarrow AC(0\text{–}7)$, $FGO \leftarrow 0$
SKI	pB_9:	If $(FGI = 1)$ then $(PC \leftarrow PC + 1)$
SKO	pB_8:	If $(FGO = 1)$ then $(PC \leftarrow PC + 1)$
ION	pB_7:	$IEN \leftarrow 1$
IOF	pB_6:	$IEN \leftarrow 0$

Designing control logic

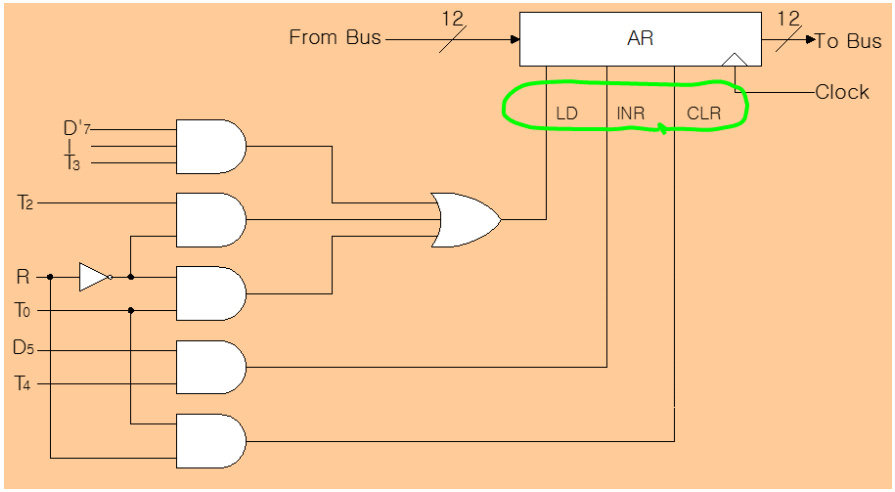
- Register control example: AR
  - Control inputs of AR: ***LD, INR, CLR***
  - Find all the statements that change the AR



$R'T_0: AR \leftarrow PC$
$R'T_2: AR \leftarrow IR(0-11)$
$D_7'IT_3: AR \leftarrow M[AR]$
$RT_0: AR \leftarrow 0$
$D_5T_4: AR \leftarrow AR + 1$

- Control functions

$$LD(AR) = R'T_0 + R'T_2 + D_7'IT_3$$
$$CLR(AR) = RT_0$$
$$INR(AR) = D_5T_4$$



Designing control logic (Up to/From here session 9/10)

- Memory control example: READ
  - Control inputs of memory: ***READ, WRITE***
  - Find all the statements that specify a ***read operation***
  - Control function

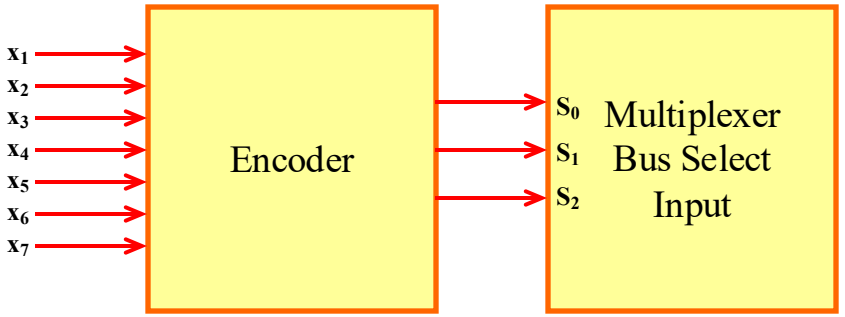
$M[AR] \leftarrow ?$

$? \leftarrow M[AR]$

$$READ = R'T_1 + D_7'IT_3 + (D_0 + D_1 + D_2 + D_6)T_4$$

Designing control logic

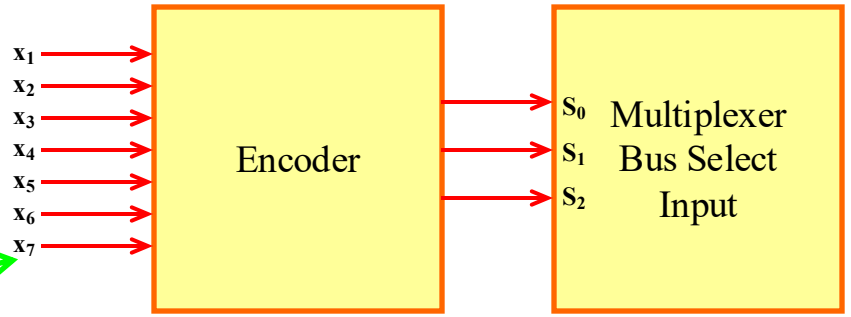
- Bus control
  - Encoder for bus selection:
    - $S_0 = x_1 + x_3 + x_5 + x_7$
    - $S_1 = x_2 + x_3 + x_6 + x_7$
    - $S_2 = x_4 + x_5 + x_6 + x_7$
  - $x_1 = 1$: $Bus \leftarrow AR = Find\ ? \leftarrow AR$
    - $D_4T_4 : PC \leftarrow AR$
      $D_5T_5 : PC \leftarrow AR$
    - Control function : $x_1 = D_4T_4 + D_5T_5$
  - $x_2 = 1$: $Bus \leftarrow PC = Find\ ? \leftarrow PC$



Designing control logic

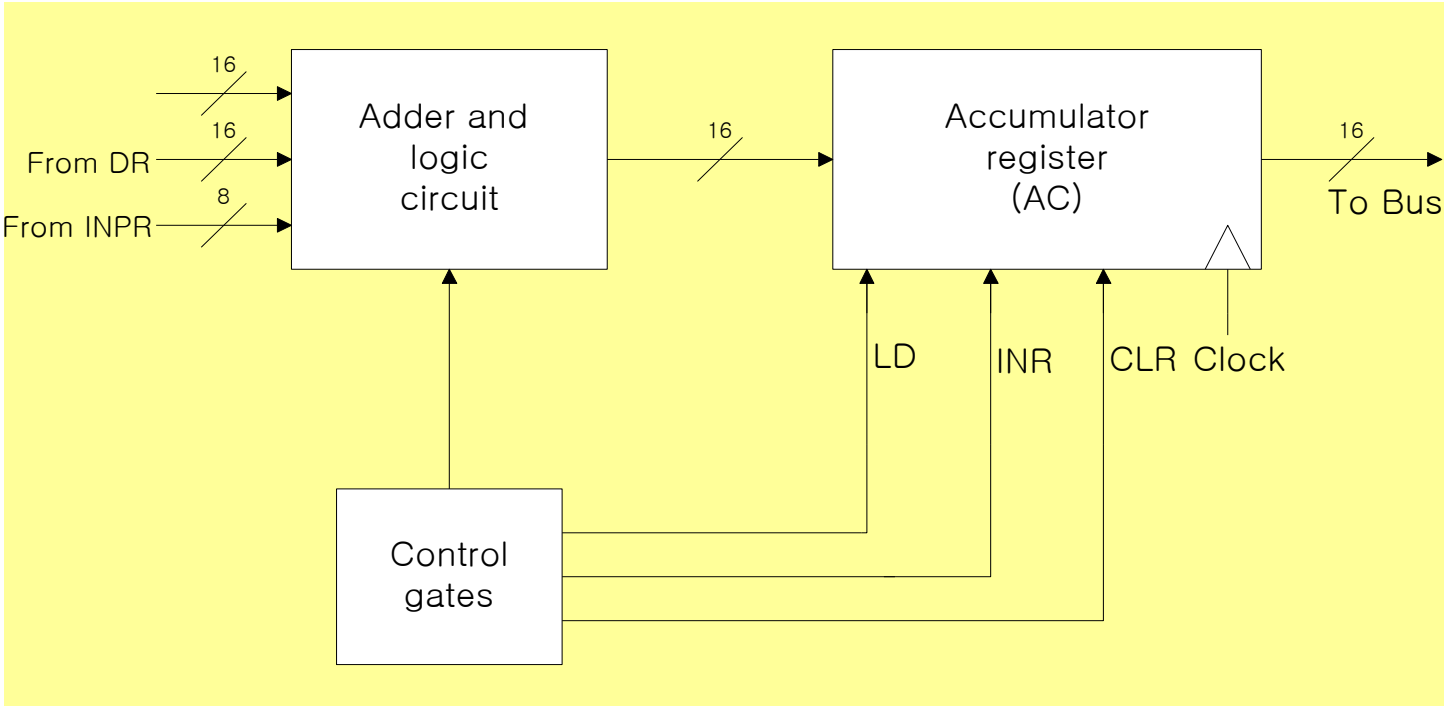
- Bus control
  - Encoder for Bus Selection :
    - $S_0 = x_1 + x_3 + x_5 + x_7$
    - $S_1 = x_2 + x_3 + x_6 + x_7$
    - $S_0 = x_4 + x_5 + x_5 + x_7$
  - $x_7 = 1$: $Bus \leftarrow Memory = Find\ ? \leftarrow M[AR]$
    - Same as memory read
    - Control function

$$x_7 = R'T_1 + D_7'IT_3 + (D_0 + D_1 + D_2 + D_6)T_4$$



Design of accumulator logic

- Circuits associated with AC



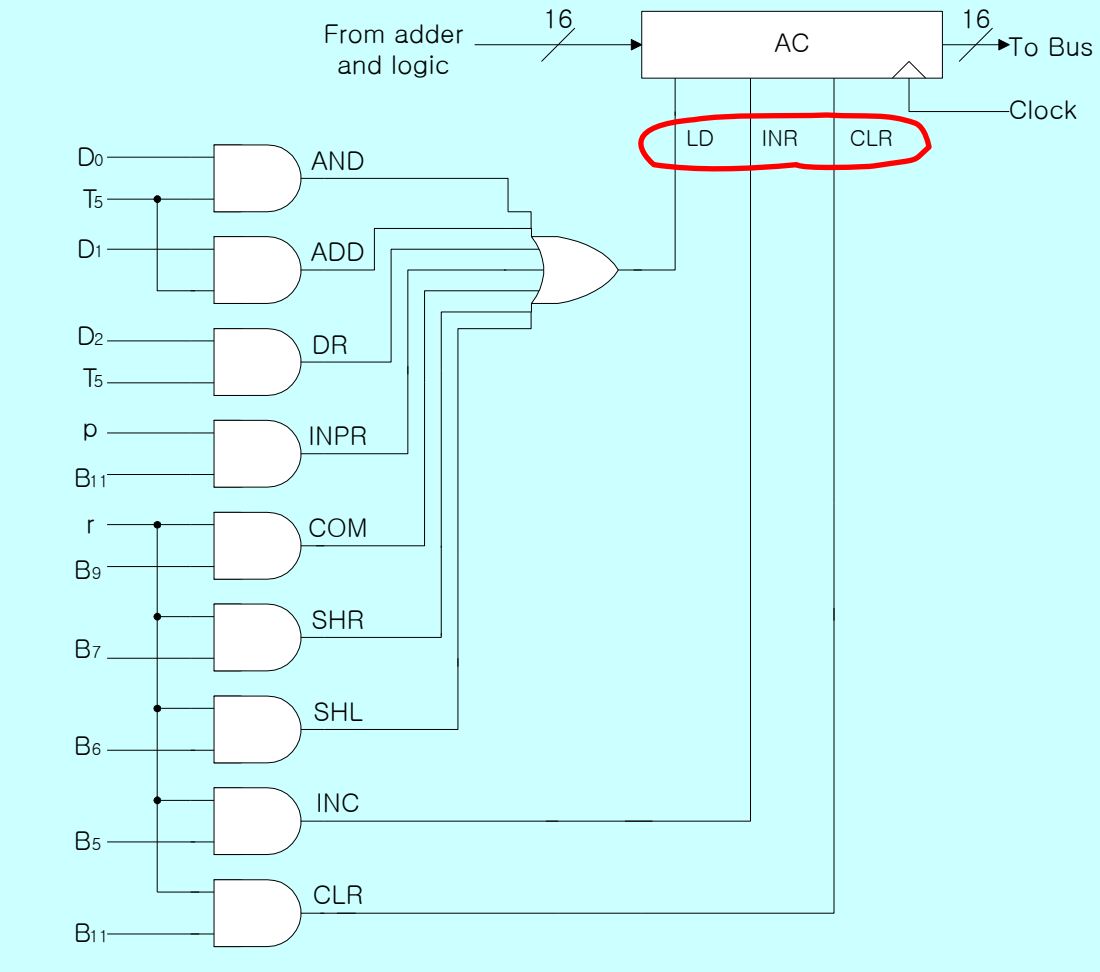
Design of accumulator logic

- Control of AC
  - Find the statement that change the AC!

$AC \leftarrow ?$

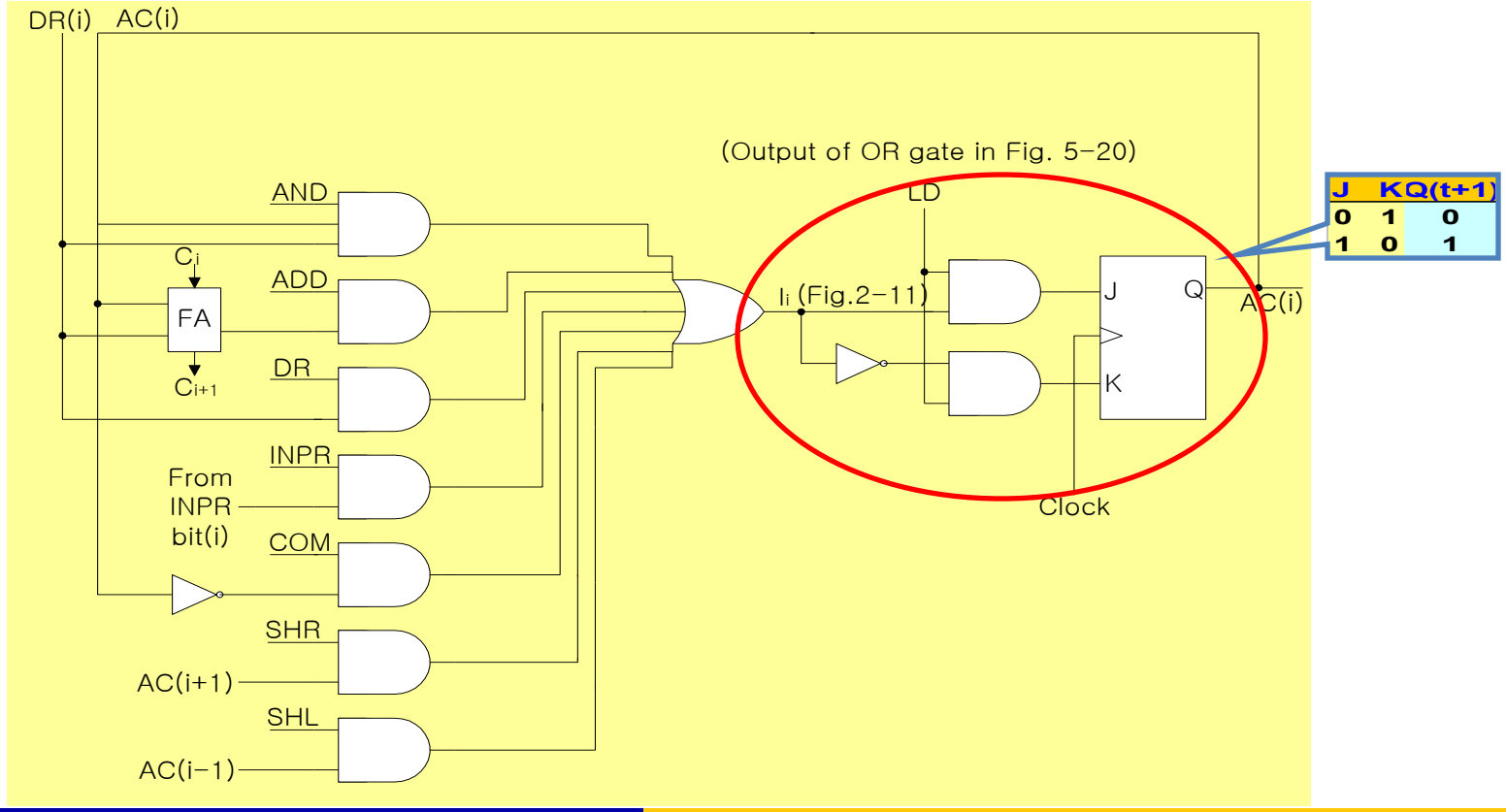
$D_0T_5 : AC \leftarrow AC \wedge DR$
$D_1T_5 : AC \leftarrow AC + DR$
$D_2T_5 : AC \leftarrow DR$
$pB_{11} : AC(0-7) \leftarrow INPR$
$rB_9 : AC \leftarrow \overline{AC}$
$rB_7 : AC \leftarrow shr\ AC, AC(15) \leftarrow E$
$rB_6 : AC \leftarrow shr\ AC, AC(0) \leftarrow E$
$rB_{11} : AC \leftarrow 0$
$rB_5 : AC \leftarrow AC + 1$

LD
CLR
INR



Design of accumulator logic

- Adder and logic circuit ***(16bit)***



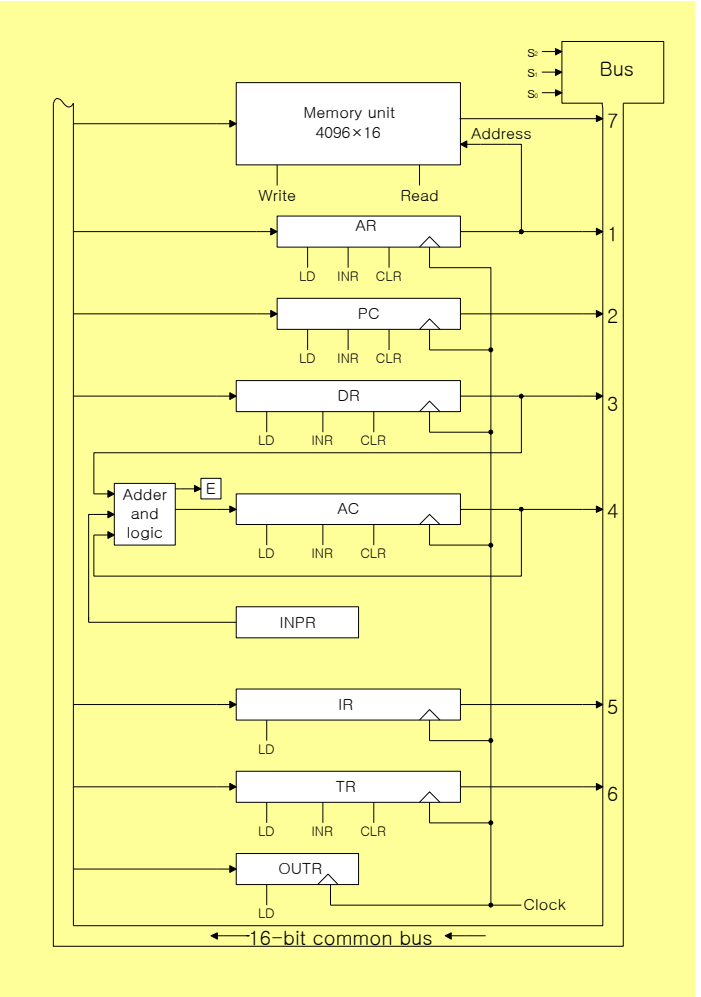
Mano machine

- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17,18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*

Integration !

Mano machine

- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17,18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*



Mano machine

- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17,18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*

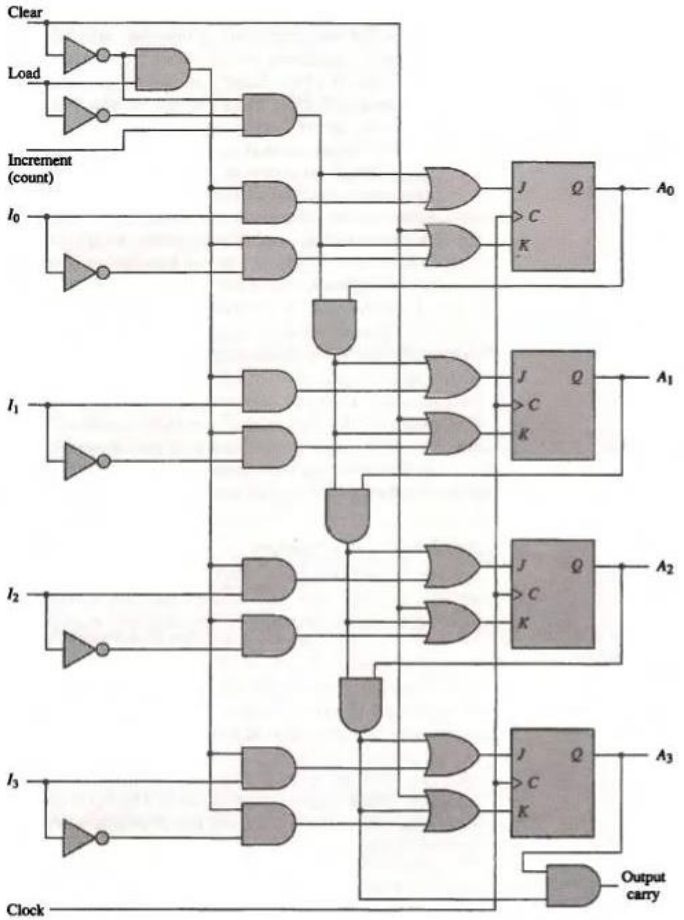


Figure 2-11 4-bit binary counter with parallel load and synchronous clear.

Mano machine

- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17,18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*

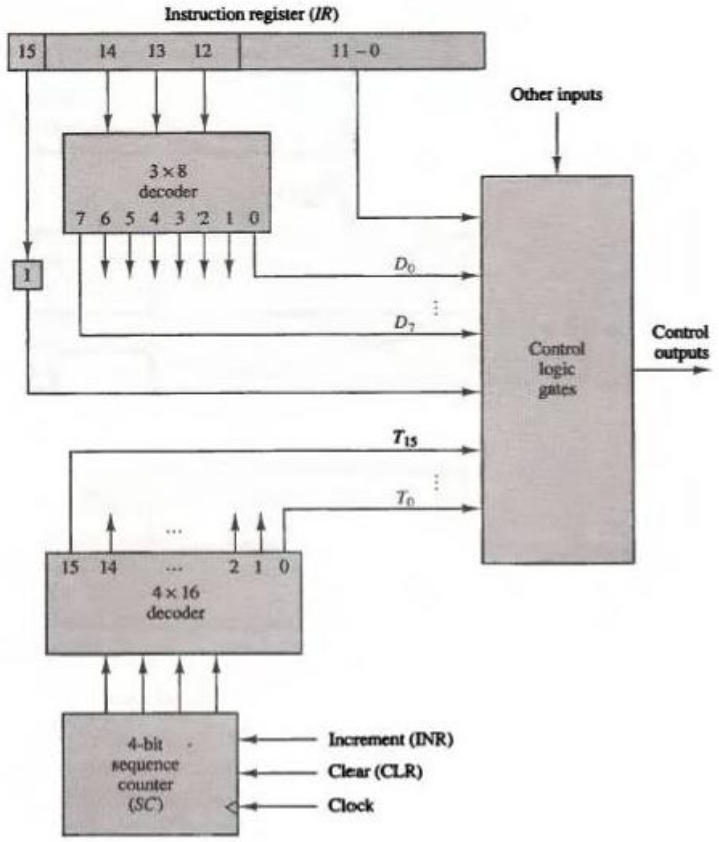


Figure 5-6 Control unit of basic computer.

Mano machine

- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17, 18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*

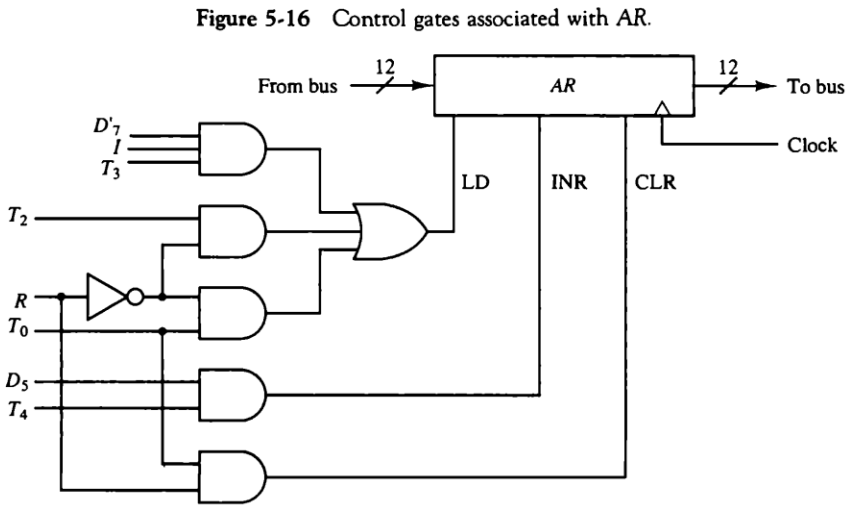


Figure 5-16 Control gates associated with AR.

Mano machine

- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17, 18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*

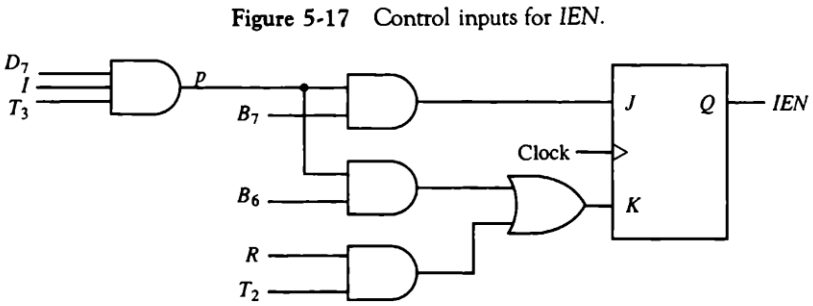
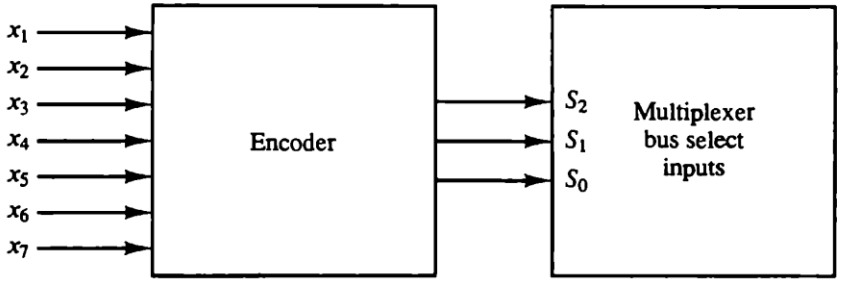


Figure 5-17 Control inputs for *IEN*.

Mano machine

- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17, 18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*

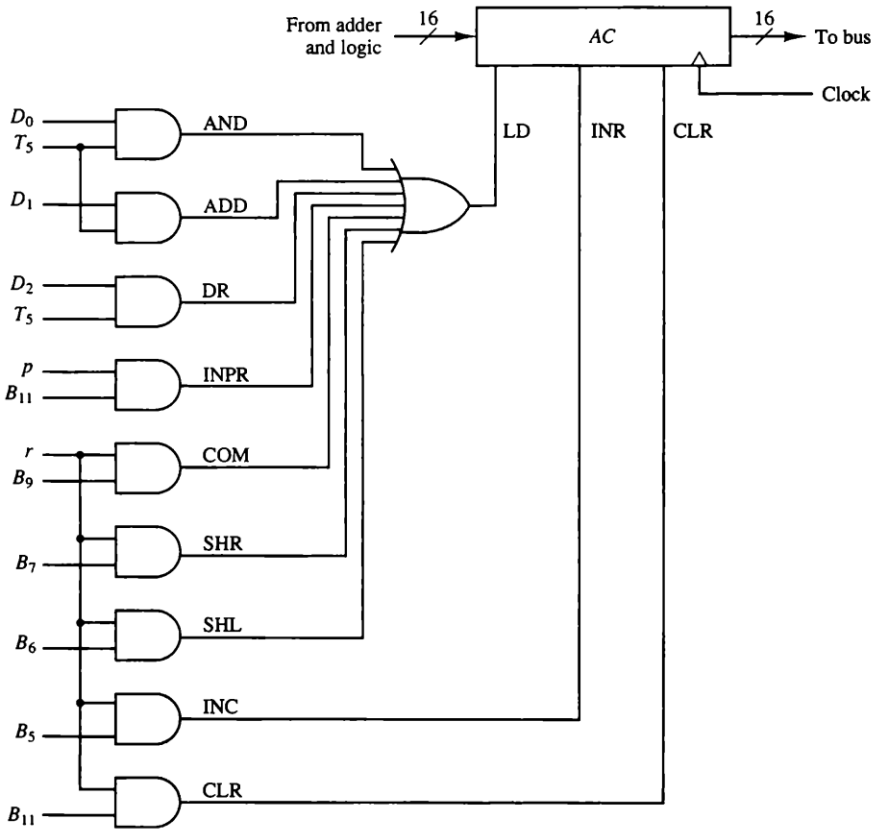
Figure 5-18 Encoder for bus selection inputs.



Mano machine

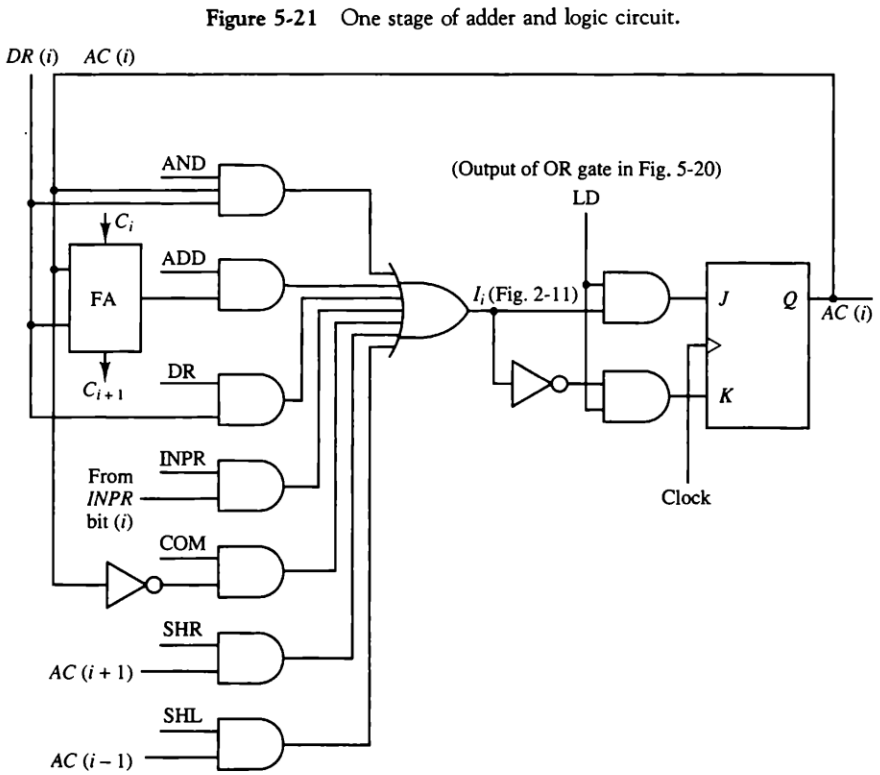
- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17, 18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*

Figure 5-20 Gate structure for controlling the LD, INR, and CLR of AC.



Mano machine

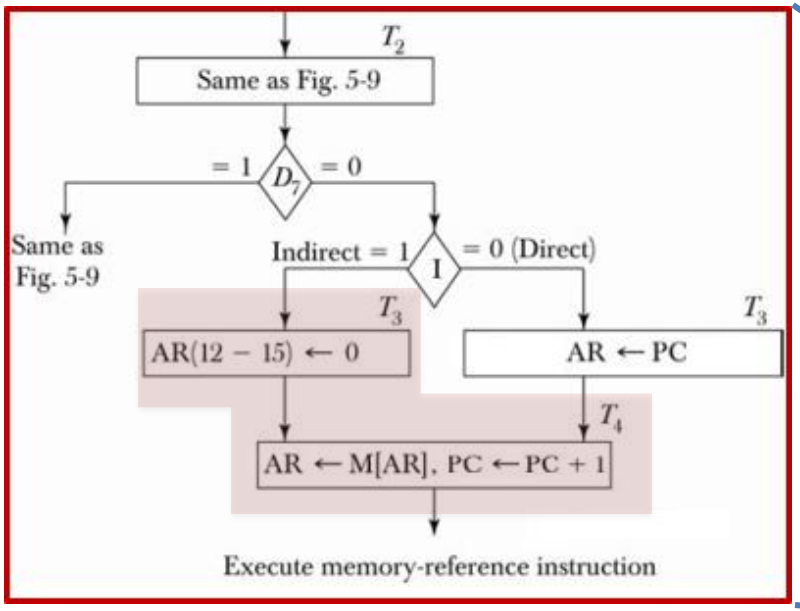
- Fig. 5-4 : Common Bus *(p.130)*
- Fig. 2-11 : Register *(p. 59)*
- Fig. 5-6 : Control Unit *(p. 137)*
- Fig. 5-16, 17, 18 : Control Logic Gate *(p.161- 163)*
  - Control inputs of all components in Fig. 5-4
  - Register, Memory, F/Fs, Bus Selection
- Fig. 5-20 : AC control *(p.165)*
- Fig. 5-21 : Adder and Logic *(p.166)*



In-class assignment

- The memory unit of the basic computer shown in Fig. 5-3 is to be changed to a 65,536 × 16 memory, requiring an address of 16 bits. The instruction format of a memory-reference instruction shown in Fig. 5-5(a) remains the same for $I = 1$ (indirect address) with the address part of the instruction residing in positions 0 through 11. But when $I = 0$ (direct address), the address of the instruction is given by the 16 bits in the next word following the instruction. Modify the microoperations during time T_2, T_3, (and T_4 if necessary) to conform with this configuration.

In-class assignment: answer



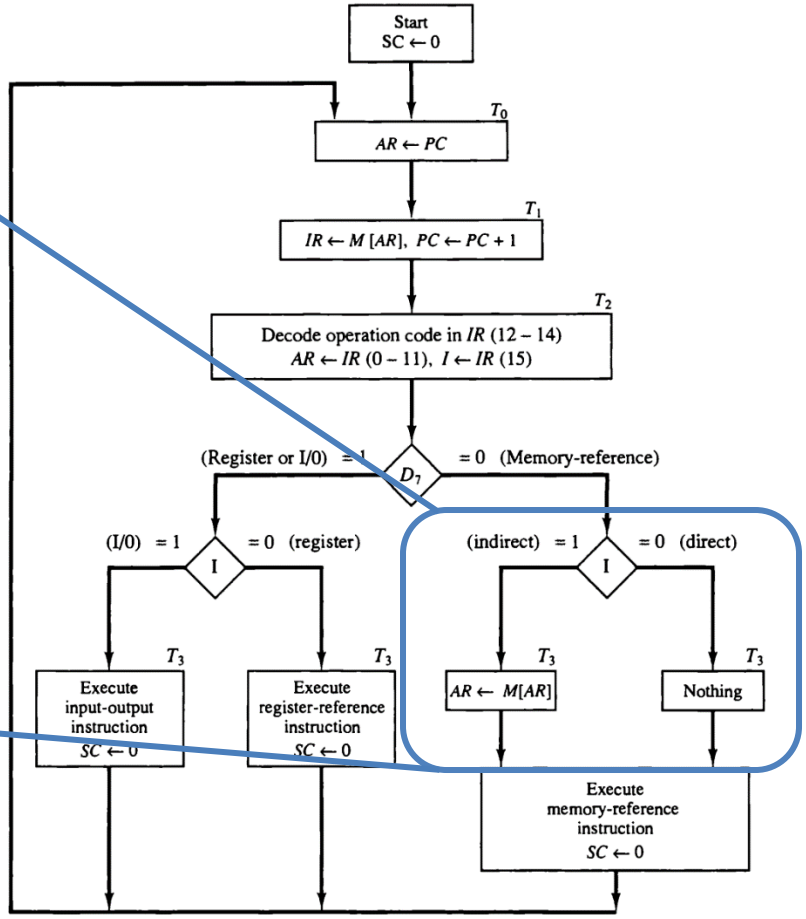


Figure 5-9 Flowchart for instruction cycle (initial configuration).

What is the problem with the above answer?

Control unit (Up to/From here session 10/11)

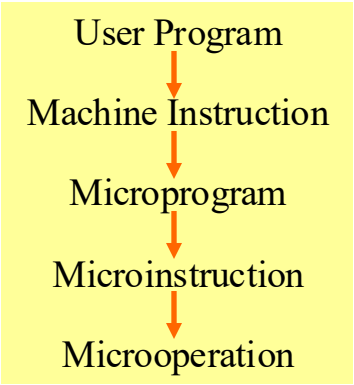
- Control word
  - The control variables at any given time can be represented by a string of 1's and 0's
- Microprogrammed control unit
  - A control unit whose binary control variables are stored in memory (*control memory*)

Control memory

- Microinstruction: *Control word in control memory*
  - The microinstruction specifies one or more microoperations
- Microprogram
  - A sequence of microinstruction
    - Dynamic microprogramming: *Control memory =* RAM
      - RAM can be used for writing (*to change a writable control memory*)
      - Microprogram is loaded initially from an auxiliary memory such as a magnetic disk
    - Static microprogramming: *Control memory =* ROM
      - Control words in ROM are made permanent during the hardware production

Control memory

- Microprogrammed control organization
  - 1) Control memory
    - A memory is part of a control unit: *Microprogram is stored*
    - Computer memory (*employs a microprogrammed control unit*)
      - Main memory: For storing user program (*machine instruction/data*)
      - Control memory: For storing microprogram (*microinstruction*)



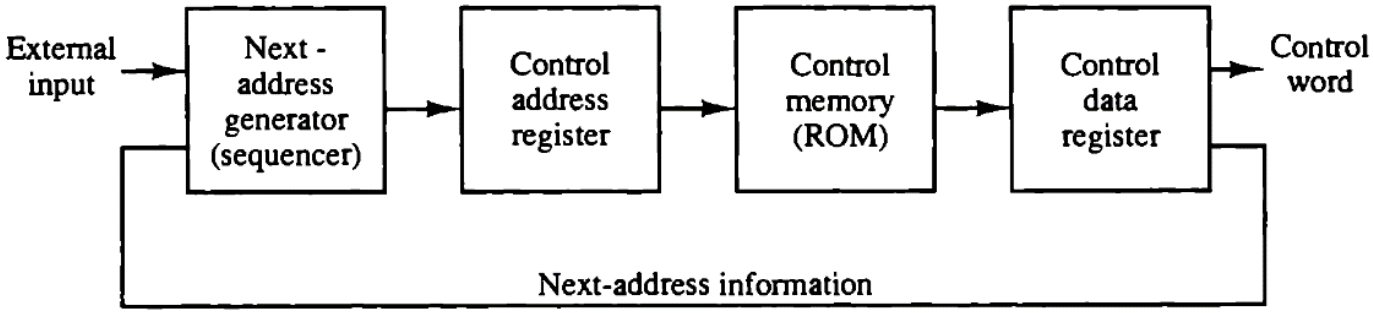
Control memory

- 2) Control address register
  - Specify the address of the microinstruction
- 3) Sequencer (= *Next address generator*)
  - Determine the address sequence that is read from control memory
  - Next address of the next microinstruction can be specified several way depending on the sequencer input: *p. 217, [1, 2, 3, and 4] of Mano's book*
- 4) Control data register (= *Pipeline register*)
  - Hold the microinstruction read from control memory
  - Allows the execution of the microoperations specified by the control word ***simultaneously*** with the generation of the next microinstruction

Control memory

- RISC architecture concept
  - RISC (Reduced Instruction Set Computer) system use hardwired control rather than microprogrammed control

Figure 7-1 Microprogrammed control organization.



Address sequencing

- Steps that the control must undergo during the execution of one instruction
  - The power is turned on in the computer
  - Fetch routine
    - Initial address is loaded into the control address register
      - Usually the address of first microinstruction that activates the instruction fetch routine
    - Fetch routine maybe sequenced by incrementing the control address register through the rest of its microinstructions
    - At the end of fetch routine the instruction is in the IR register of the computer

Address sequencing

- Steps that the control must undergo during the execution of one instruction
  - Determine the effective address
    - The effective address computation routine can be reached through a branch microinstruction
      - Direct or indirect?
    - At the end, the address of operand is loaded in the memory address register
  - Executing the instruction
    - Each instruction has its own microprogram routine

Address sequencing

- Address sequencing = sequencer: Next address generator
  - Selection of address for control memory
- Routine
  - Microinstruction are stored in control memory *in a group*
- Mapping
  - Instruction code ⟶ Address in control memory (*where routine is located*)

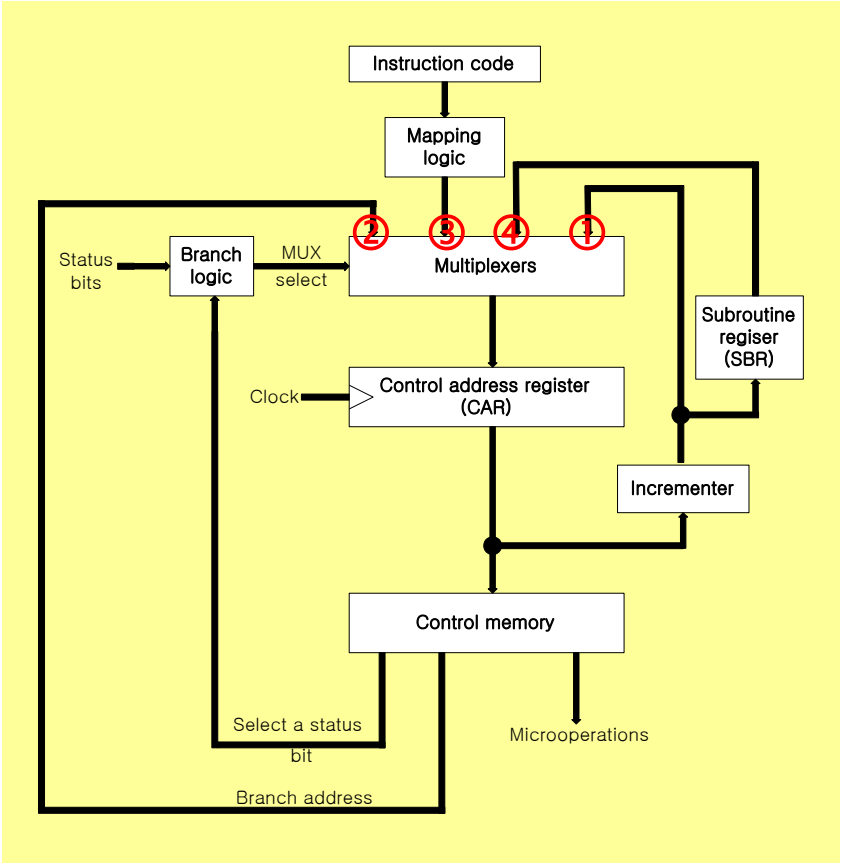
Address sequencing

- Address sequencing capabilities: ***control memory address***
  - 1) Incrementing of the control address register
  - 2) Unconditional branch or conditional branch, depending on status bit conditions
  - 3) Mapping process (*bits of the instruction address for control memory*)
  - 4) A facility for subroutine call and return

Subroutine : program used by other ROUTINES

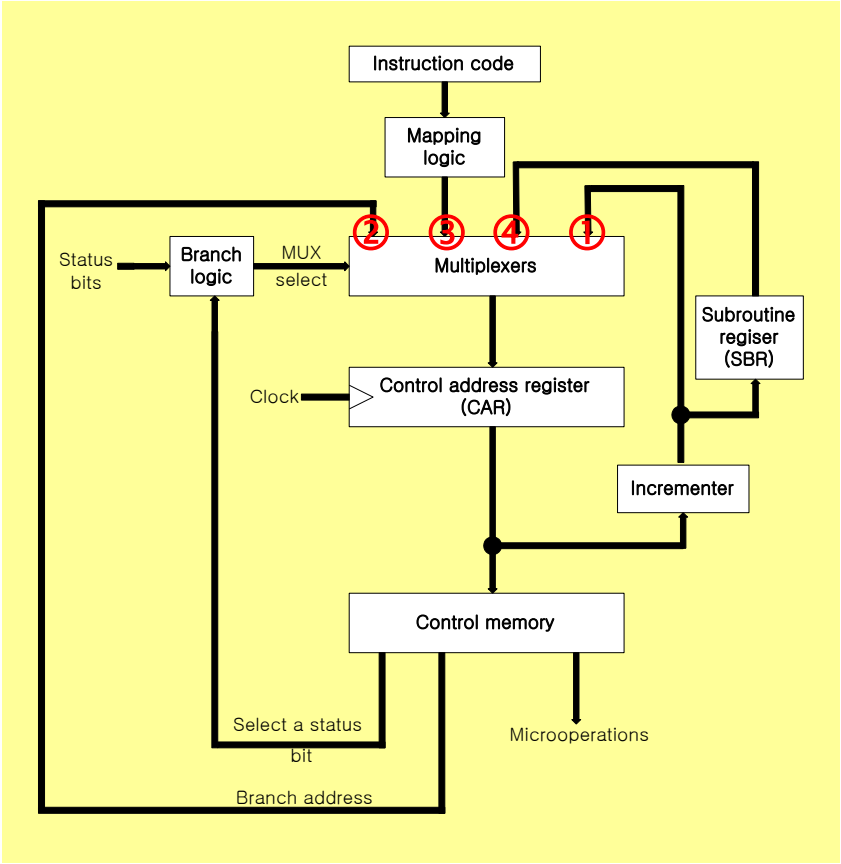
Address sequencing

- Selection of address for control memory
  - Multiplexer
    - ① CAR Increment
    - ② JMP/CALL
    - ③ Mapping
    - ④ Subroutine return



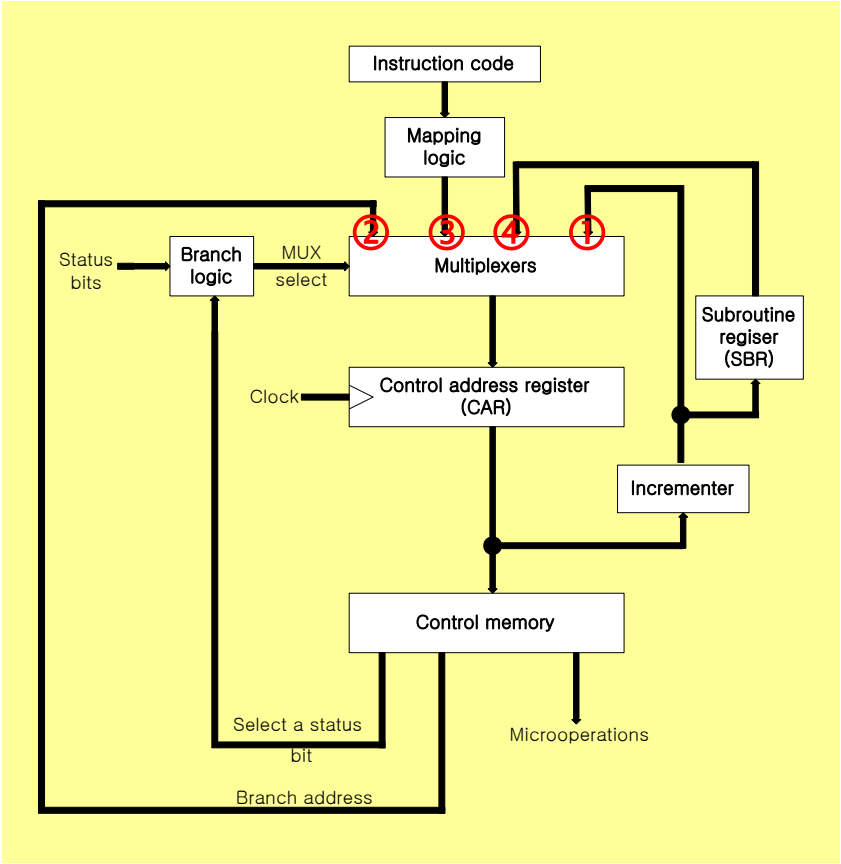
Address sequencing

- Selection of address for control memory
  - CAR: Control address register
    - CAR receive the address from 4 different paths
      - 1) Incrementer
      - 2) Branch address from control memory
      - 3) Mapping Logic
      - 4) SBR: Subroutine Register



Address sequencing

- SBR: Subroutine register
  - Return address can not be stored in ROM
  - Return address for a subroutine is stored in SBR
- Conditional branching
  - Status bits
    - Control the conditional branch decisions generated in the ***Branch Logic***



Address sequencing

- Conditional branching
  - Branch logic
    - Test the specified condition and branch to the indicated address if the condition is met; otherwise, the control address register is just incremented
  - The actual circuit of status bit test and branch logic
    - Composed of 4 X 1 Mux and input logic

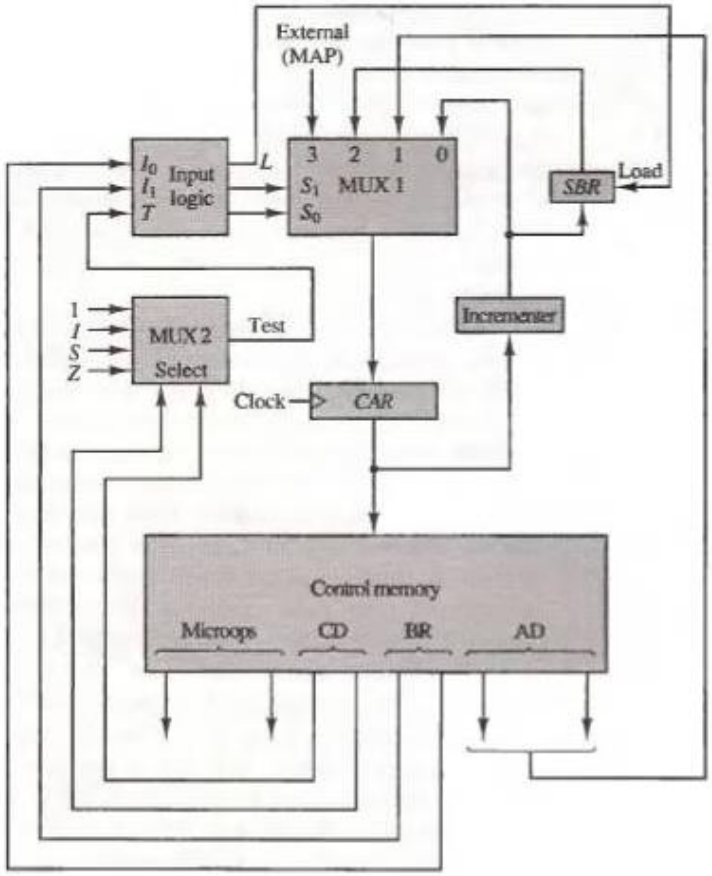
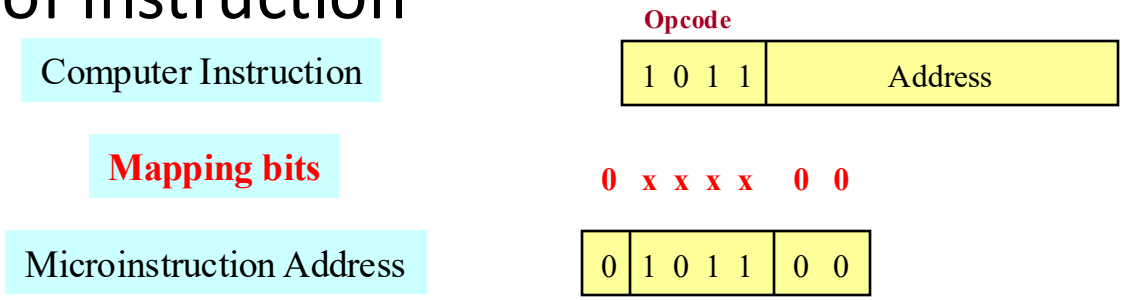


Figure 7-8 Microprogram sequencer for a control memory.

Address sequencing

- Mapping of instruction



  - 4 bit opcode: specify up to 16 distinct instruction
  - Mapping process: Converts *the 4-bit Opcode* to *a 7-bit control memory address*
    - 1) Place a "0" in the most significant bit of the address
    - 2) Transfer 4-bit operation code bits
    - 3) Clear the two least significant bits of the CAR (*In other words, it can accommodate 4 microinstructions*)
  - Mapping function: Implemented by *Mapping ROM* or *PLD*

Address sequencing

- Subroutine
  - Subroutines are programs that are used by other routines
    - Subroutine can be called from any point within the main body of the microprogram
  - Microinstructions can be saved by subroutines that use common section of microcode
    - ***Example)*** Subroutine to obtain effective address of operand in memory reference command
      - **INDRCT** (*where FETCH and INDRCT are Subroutine*)
  - Subroutine must have a provision for
    - Storing the return address during a subroutine call
    - Restoring the address during a subroutine return
      - Last-In First Out (LIFO) register stack

Address sequencing

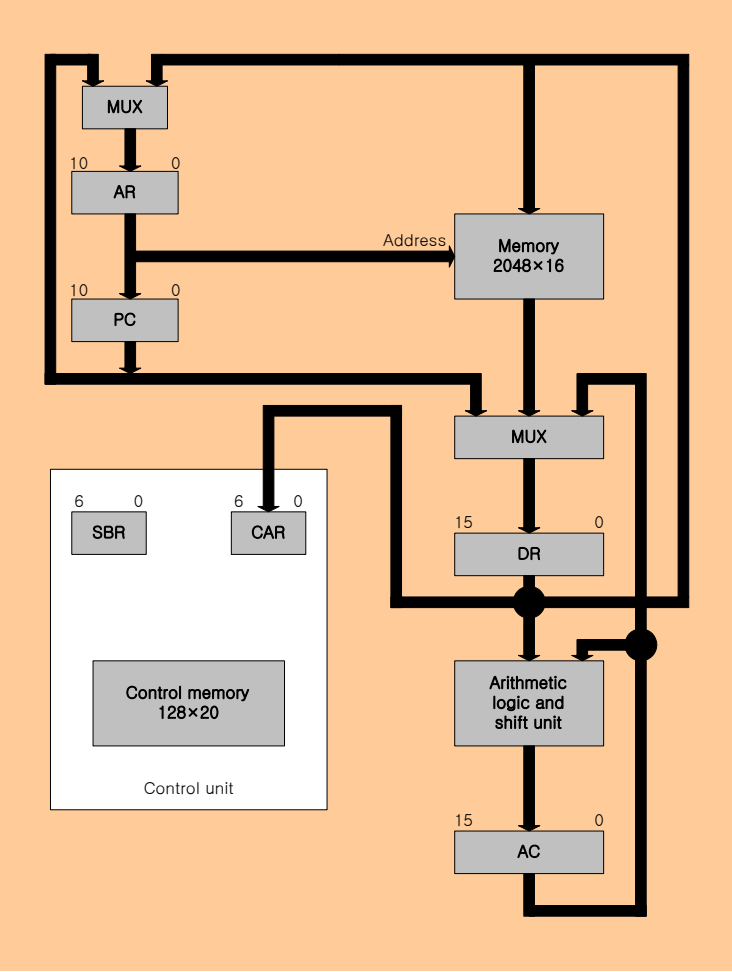
- Subroutine examples

TABLE 7-2 Symbolic Microprogram (Partial)

Label	Microoperations	CD	BR	AD
	ORG 0			
ADD:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ADD	U	JMP	FETCH
	ORG 4			
BRANCH:	NOP	S	JMP	OVER
	NOP	U	JMP	FETCH
OVER:	NOP	I	CALL	INDRCT
	ARTPC	U	JMP	FETCH
	ORG 8			
STORE:	NOP	I	CALL	INDRCT
	ACTDR	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 12			
EXCHANGE:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ACTDR, DRTAC	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 64			
FETCH:	PCTAR	U	JMP	NEXT
	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	
INDRCT:	READ	U	JMP	NEXT
	DRTAR	U	RET	

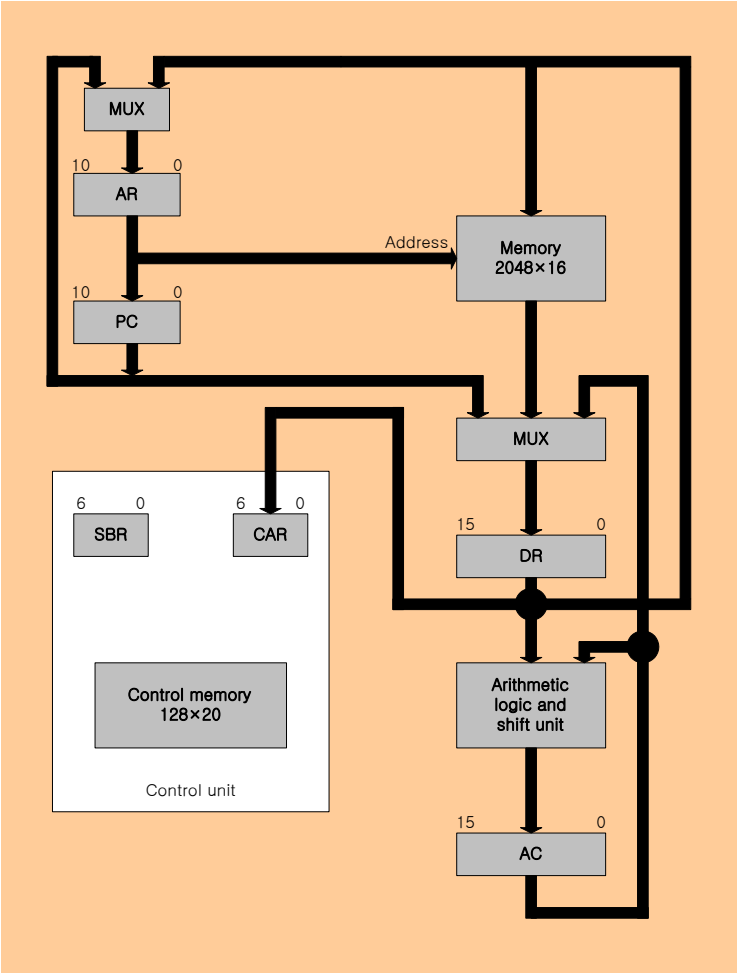
Microprogram example

- Computer configuration
  - Two memories
    - Main memory
      - *Instruction*/*data*
    - Control memory
      - *Microprogram*
  - Data written to memory come from DR, and Data read from memory can go only to DR



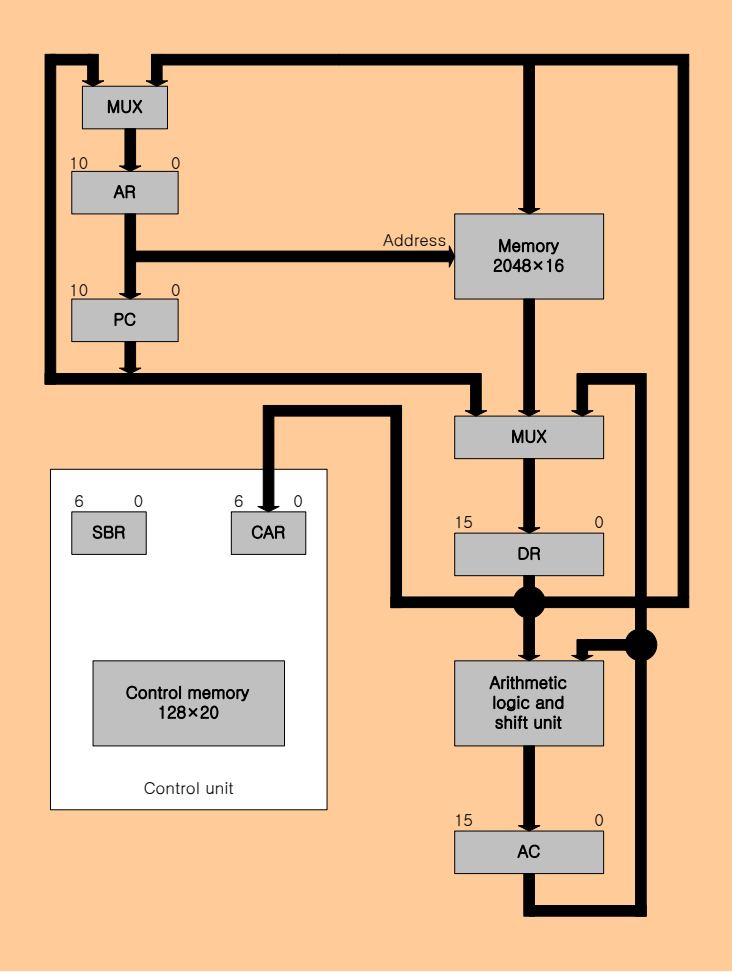
Microprogram example

- Computer configuration
  - Four CPU registers and ALU: DR, AR, PC, AC, ALU
    - DR can receive information from AC, PC, or Memory (*selected by MUX*)
    - AR can receive information from PC or DR (*selected by MUX*)
    - PC can receive information only from AR
    - ALU performs microoperation with data from AC and DR (*Results stored in AC*)



Microprogram example

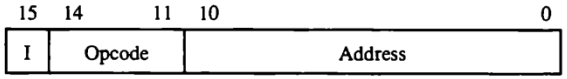
- Computer configuration
  - Two control unit registers
    - SBR
    - CAR



Microprogram example (Up to/From here session 11/12)

- Instruction format
  - Instruction format
    - I: 1 bit for indirect addressing
    - Opcode: 4 bit operation code
    - Address: 11 bit address for system memory

Figure 7-5 Computer instructions.



(a) Instruction format

  - Computer instruction
    - 16 instructions available, only 4

Symbol	Opcode	Description
ADD	0000	$AC \leftarrow AC + M[EA]$
BRANCH	0001	If $(AC < 0)$ then $(PC \leftarrow EA)$
STORE	0010	$M[EA] \leftarrow AC$
EXCHANGE	0011	$AC \leftarrow M[EA], M[EA] \leftarrow AC$

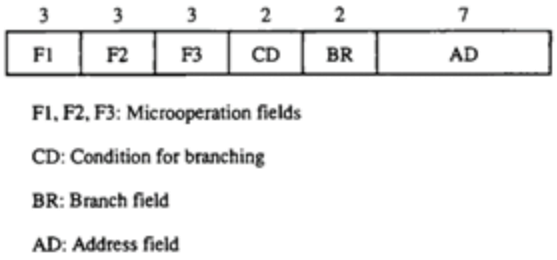
EA is the effective address

(b) Four computer instructions

Microprogram example

- Microinstruction format
  - Three bit microoperation fields: F1, F2, F3
    - Total 21 microoperation
    - Three microoperation can be run at same time
      - If we have less than three, fill some fields with 000 (no operation)
    - Two or more conflicting microoperations can not be specified simultaneously
      - ***Example)*** 010 001 000

        Clear AC to 0 and subtract DR from AC at the same time



Non-confilicing

000 100 101

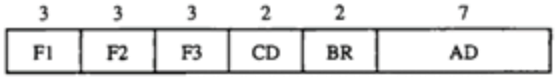
$DR \leftarrow M[AR]$ with F2 = 100

and $PC \leftarrow PC + 1$ with F3 = 101

    - Symbol **DRTAC** (F1 = 100)
      - Stand for a transfer from DR to AC (*T = to*)

Microprogram example

- Microinstruction format
  - Three bit microoperation fields: F1, F2, F3
    - Total 21 microoperation



F1, F2, F3: Microoperation fields

CD: Condition for branching

BR: Branch field

AD: Address field

TABLE 7-1 Symbols and Binary Code for Microinstruction Fields

F1	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC + DR$	ADD
010	$AC \leftarrow 0$	CLRAC
011	$AC \leftarrow AC + 1$	INCAC
100	$AC \leftarrow DR$	DRTAC
101	$AR \leftarrow DR(0\text{–}10)$	DRTAR
110	$AR \leftarrow PC$	PCTAR
111	$M[AR] \leftarrow DR$	WRITE

F2	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC - DR$	SUB
010	$AC \leftarrow AC \vee DR$	OR
011	$AC \leftarrow AC \wedge DR$	AND
100	$DR \leftarrow M[AR]$	READ
101	$DR \leftarrow AC$	ACTDR
110	$DR \leftarrow DR + 1$	INCDR
111	$DR(0\text{-}10) \leftarrow PC$	PCTDR

F3	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC \oplus DR$	XOR
010	$AC \leftarrow \overline{AC}$	COM
011	$AC \leftarrow$ shl AC	SHL
100	$AC \leftarrow$ shr AC	SHR
101	$PC \leftarrow PC + 1$	INCPC
110	$PC \leftarrow AR$	ARTPC
111	Reserved	

CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
01	$DR(15)$	I	Indirect address bit
10	$AC(15)$	S	Sign bit of AC
11	$AC = 0$	Z	Zero value in AC

BR	Symbol	Function
00	JMP	$CAR \leftarrow AD$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
01	CALL	$CAR \leftarrow AD$, $SBR \leftarrow CAR + 1$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
10	RET	$CAR \leftarrow SBR$ (Return from subroutine)
11	MAP	$CAR(2\text{–}5) \leftarrow DR(11\text{–}14)$, $CAR(0,1,6) \leftarrow 0$

Microprogram example

- Microinstruction format
  - Two bit condition fields: CD
    - 00: Unconditional branch,
      - **U** = always 1
    - 01: Indirect address bit
      - **I** = DR(15)
    - 10: Sign bit of AC
      - **S** = AC(15)
    - 11: Zero value in AC
      - **Z** = AC = 0

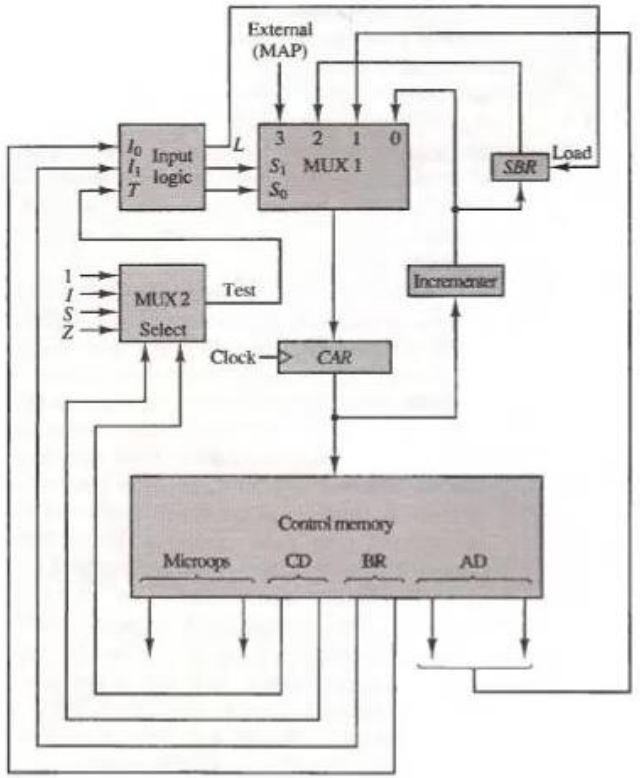


Figure 7-8 Microprogram sequencer for a control memory.

Microprogram example

- Microinstruction format
  - Two bit branch fields: BR
    - 00: **JMP**
      - Condition = 0 : 1 $CAR \leftarrow CAR + 1$
      - Condition = 1 : 2 $CAR \leftarrow AD$
    - 01: **CALL**
      - Condition = 0 : 1 $CAR \leftarrow CAR + 1$
      - Condition = 1 : 2 $CAR \leftarrow AD, SBR \leftarrow CAR + 1$
    - 10: **RET** 3 $CAR \leftarrow SBR$
    - 11: **MAP** 4 $CAR(2-5) \leftarrow DR(11-14), CAR(0,1,6) \leftarrow 0$
  - Seven bit address fields: AD
    - 128 word: 128 X 20 bit

Save Return Address

Restore Return Address

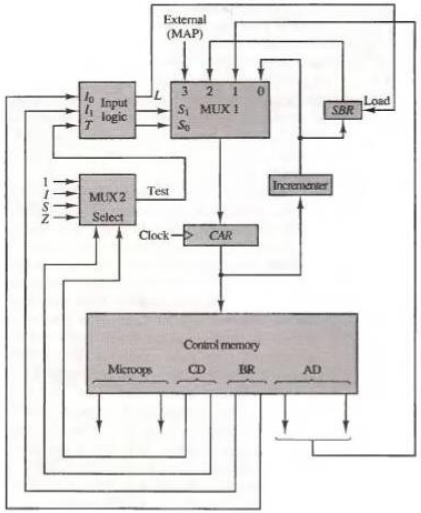


Figure 7-8 Microprogram sequencer for a control memory.

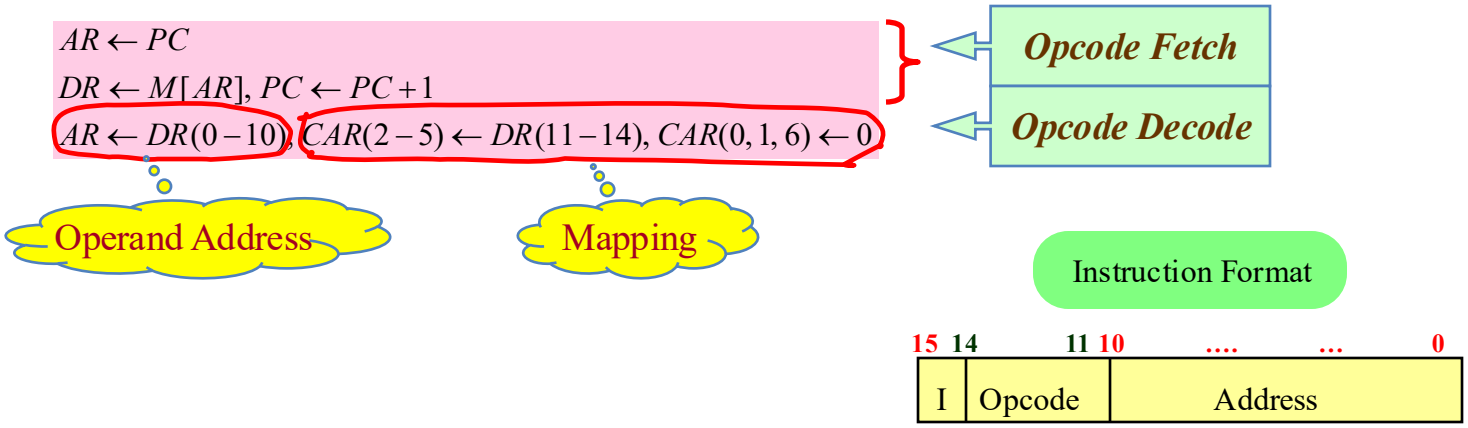
Microprogram example

- Symbolic microinstruction
  - ① Label field: Terminated with a colon (:)
  - ② Microoperation field: one, two, or three symbols, separated by commas
  - ③ CD Field: U, I, S, or Z
  - ④ BR Field: JMP, CALL, RET, or MAP
  - ⑤ AD Field
    - a. Symbolic address: Label (= Address)
    - b. Symbol “NEXT”: next address
    - c. When symbol “RET” or “MAP” used in microoperation: AD field = 0000000
  - ORG: Pseudo inst. (the origin, or the first address of routine)

①	②	③	④	⑤
Label	Microoperation	CD	BR	AD
	ORG 64			
FETCH:	PCTAR	U	JMP	NEXT
	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	0

Microprogram example

- Fetch (**sub**) routine
  - Memory map (128 words)
    - Address 0 to 63: Routines for the 16 instruction (*currently 4 instruction*)
    - Address 64 to 127: Any other purpose (currently subroutines: FETCH, INDRCT)



Microprogram example

- Fetch (**sub**) routine
  - Microinstruction for FETCH Subroutine

Label	Microoperation	CD	BR	AD
	ORG 64			
FETCH:	PCTAR	U	JMP	NEXT
	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	0

- Fetch subroutine: address 64

Microprogram example

- Symbolic microprogram
  - The execution of MAP microinstruction in FETCH subroutine
    - Branch to address 0**xxxx**00 (xxxx = 4 bit Opcode)
      - ADD: 0 **0000** 00 = 0
      - BRANCH: 0 **0001** 00 = 4 $AR \leftarrow M[AR]$
      - STORE : 0 **0010** 00 = 8
      - EXCHANGE : 0 **0011** 00 = 12, (16, 20, … , 60)
  - Indirect address: **I** = 1
    - Indirect addressing
      - Read the contents of the memory pointed to by the AR into the DR, and write it back to the AR
    - INDRCT subroutine

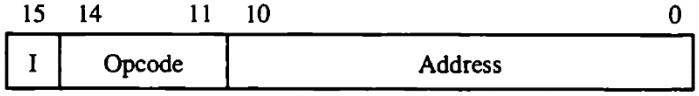
Label	Microoperation	CD	BR	AD
INDRCT:	READ	U	JMP	NEXT
	DRTAR	U	RET	0

$DR \leftarrow M[AR]$
$AR \leftarrow DR$

Microprogram example

- Execution of instruction
  - Procedure for executing ADD instruction
    - 1) When the ADD command is executed, the opcode is fetched from the FETCH subroutine, and when the MAP is executed, the MAP process branches CAR = 0 0000 00 (where Opcode = 0000)
    - 2) In the address 0 of the ADD routine, the CD bit is checked. If Indirect = 1, the effective address is written to the AR in the INDRCT subroutine

Figure 7-5 Computer instructions.



(a) Instruction format

Symbol	Opcode	Description
ADD	0000	$AC \leftarrow AC + M\,[EA]$
BRANCH	0001	If $(AC < 0)$ then $(PC \leftarrow EA)$
STORE	0010	$M\,[EA] \leftarrow AC$
EXCHANGE	0011	$AC \leftarrow M[EA], M[EA] \leftarrow AC$

EA is the effective address

(b) Four computer instructions

Microprogram example

- Execution of instruction
  - Procedure for executing ADD instruction
    - 3) Address 1 of the ADD routine reads the contents of the memory indicated by the AR and writes them to the DR.
    - 4) AC + DR is stored in AC and jumps back to the beginning of fetch routine.

TABLE 7-2 Symbolic Microprogram (Partial)

Label	Microoperations	CD	BR	AD
	ORG 0			
ADD:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ADD	U	JMP	FETCH
	ORG 4			
BRANCH:	NOP	S	JMP	OVER
	NOP	U	JMP	FETCH
OVER:	NOP	I	CALL	INDRCT
	ARTPC	U	JMP	FETCH
	ORG 8			
STORE:	NOP	I	CALL	INDRCT
	ACTDR	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 12			
EXCHANGE:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ACTDR, DRTAC	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 64			
FETCH:	PCTAR	U	JMP	NEXT
	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	
INDRCT:	READ	U	JMP	NEXT
	DRTAR	U	RET	

Microprogram example

- Binary microprogram
  - Symbolic microprogram must be translated to **binary** either by means of an assembler program or by the user
  - Control memory
    - Most microprogrammed systems use a ROM for the control memory
      - Cheaper and faster than a RAM
      - Prevent the occasional user from changing the architecture of the system

Microprogram example

- Binary microprogram

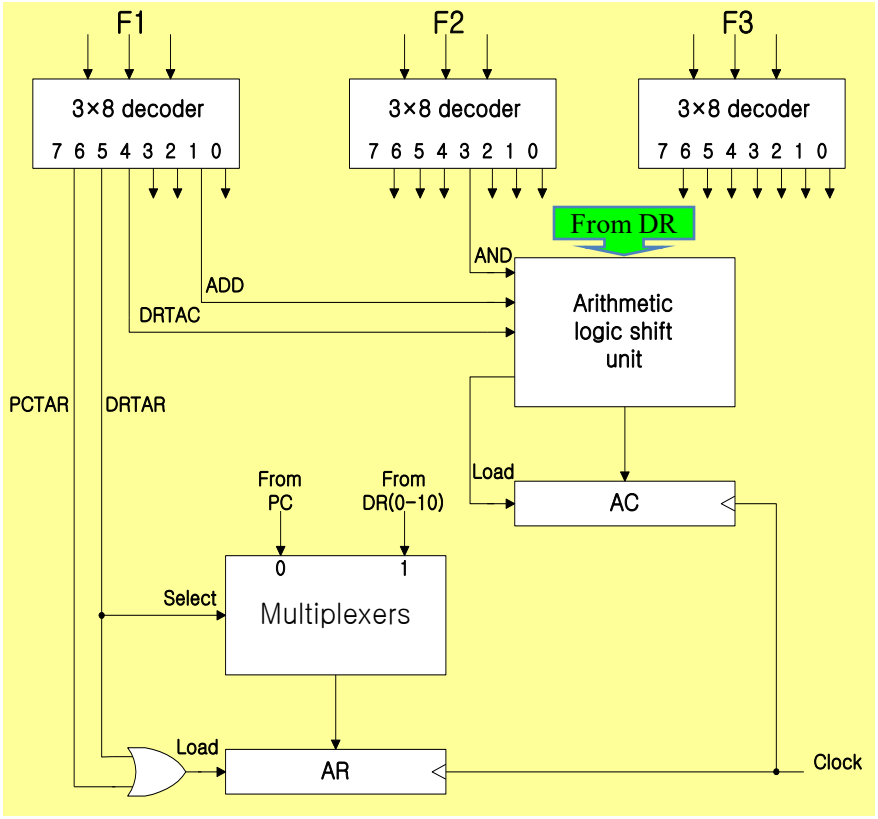
TABLE 7-3 Binary Microprogram for Control Memory (Partial)

Micro Routine	Address		Binary Microinstruction					
	Decimal	Binary	F1	F2	F3	CD	BR	AD
ADD	0	0000000	000	000	000	01	01	1000011
	1	0000001	000	100	000	00	00	0000010
	2	0000010	001	000	000	00	00	1000000
	3	0000011	000	000	000	00	00	1000000
BRANCH	4	0000100	000	000	000	10	00	0000110
	5	0000101	000	000	000	00	00	1000000
	6	0000110	000	000	000	01	01	1000011
	7	0000111	000	000	110	00	00	1000000
STORE	8	0001000	000	000	000	01	01	1000011
	9	0001001	000	101	000	00	00	0001010
	10	0001010	111	000	000	00	00	1000000
	11	0001011	000	000	000	00	00	1000000
EXCHANGE	12	0001100	000	000	000	01	01	1000011
	13	0001101	001	000	000	00	00	0001110
	14	0001110	100	101	000	00	00	0001111
	15	0001111	111	000	000	00	00	1000000
FETCH	64	1000000	110	000	000	00	00	1000001
	65	1000001	000	100	101	00	00	1000010
	66	1000010	101	000	000	00	11	0000000
INDRCT	67	1000011	000	100	000	00	00	1000100
	68	1000100	101	000	000	00	10	0000000

Design of control unit (Up to/From here session 12/13)

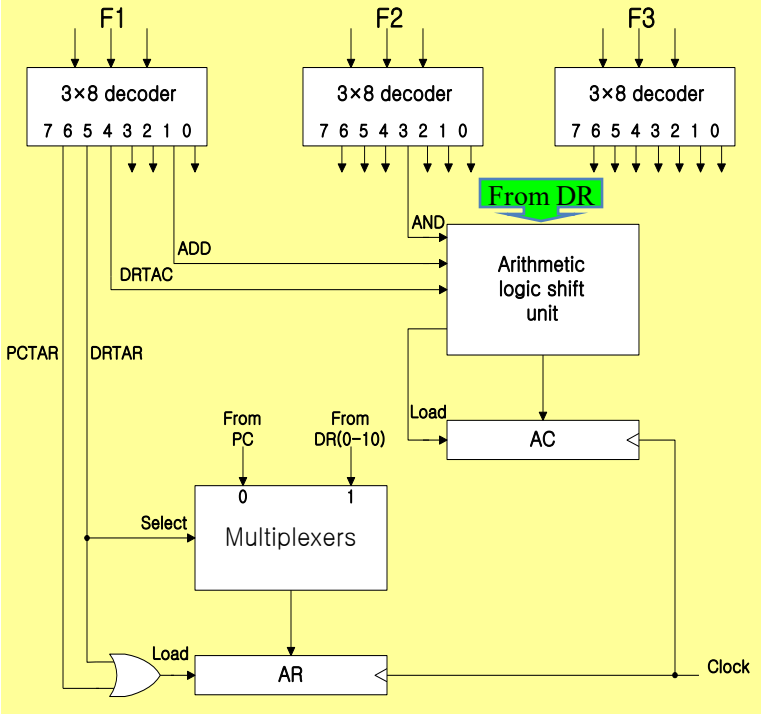
- Decoding of microinstruction fields
  - F1, F2, and F3 of microinstruction are decoded with a 3 x 8 decoder
  - Output of decoder must be connected to the proper circuit to initiate the corresponding microoperation
    - ***Example)***
      - F1 = 101 (5): **DRTAR**
      - F1 = 110 (6): **PCTAR**
        - » Output 5 and 6 of decoder F1 are connected to the load input of AR (two input of OR gate)
        - » Multiplexer select the data from DR when output 5 is active and select the data from PC when output 5 is inactive

Design of control unit



Design of control unit

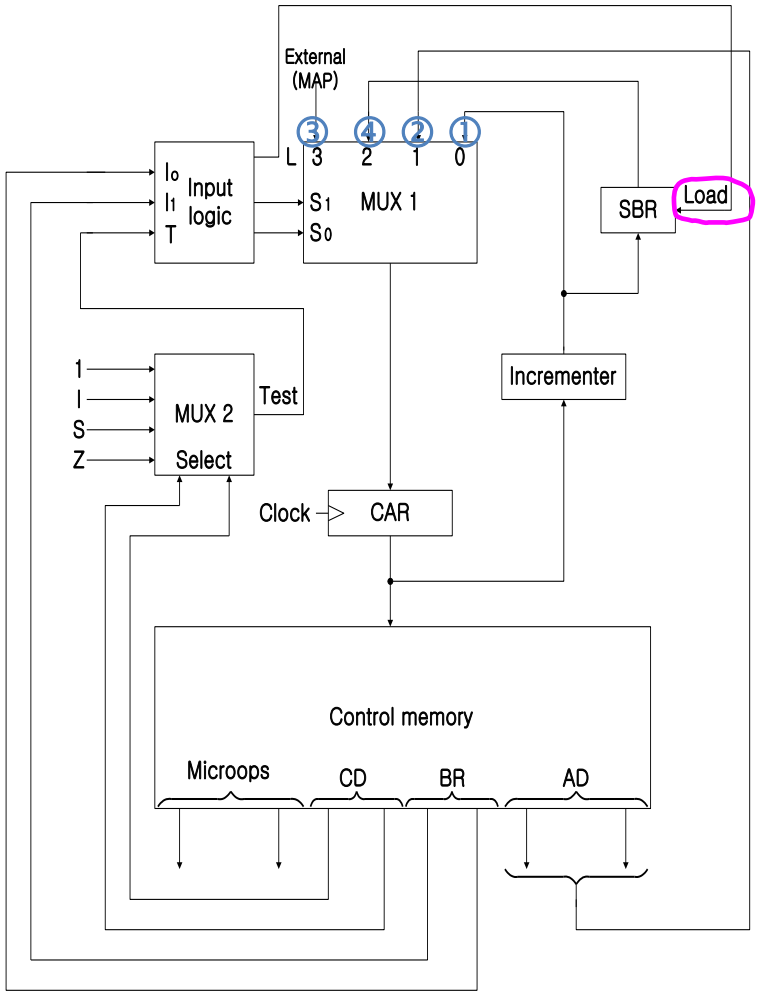
- Arithmetic logic shift unit
  - Control signal will be now come from the output of the decoders associated with the AND, ADD, and DRTAC.



Design of control unit

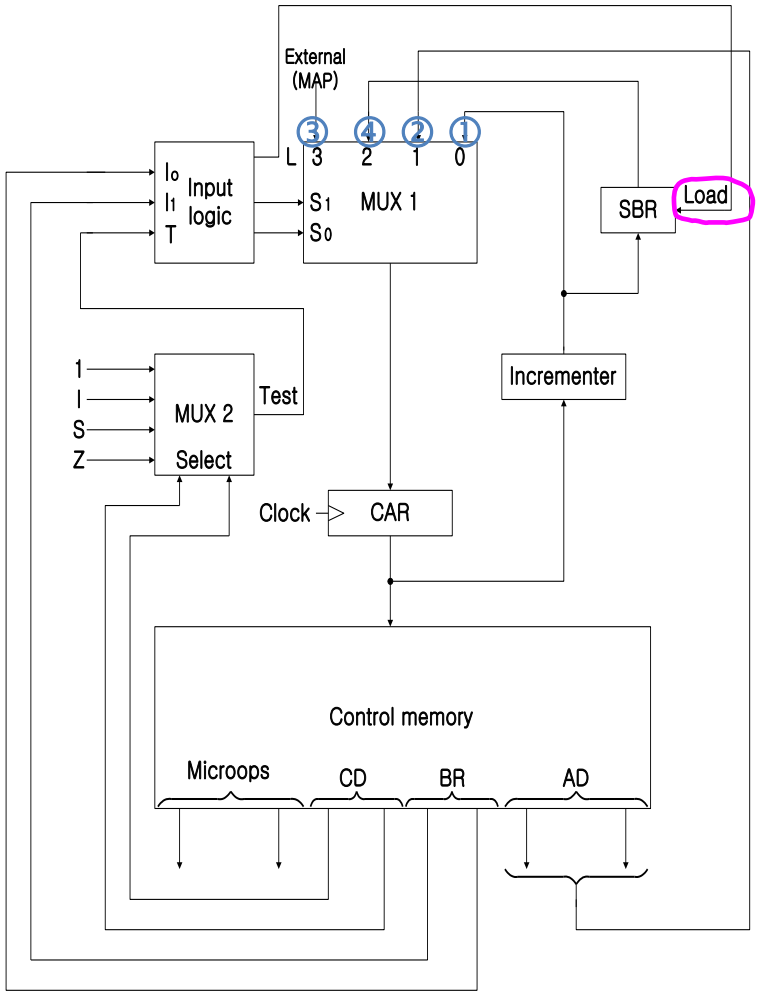
- Microprogram sequencer
  - Microprogram Sequencer select the next address for control memory
  - MUX 1
    - Select an address source and route to CAR

① CAR + 1
② JMP/CALL
③ Mapping
④ Subroutine Return



Design of control unit

- Microprogram sequencer:
  - MUX 1
    - Difference between JMP and CALL
      - JMP: AD is sent to CAR via MUX 1
      - CALL: AD is sent to CAR via MUX 1, and CAR + 1 (return address) is stored in SBR by LOAD signal at the same time



Design of control unit

- Input logic truth table
  - Input
    - I_0, I_1 from Branch bit (**BR**)
    - T from MUX 2 (**T**)
  - Output
    - MUX 1 select signal (**S_0, S_1**)

      $S1 = I_1I_0' + I_1I_0 = I_1(I_0' + I_0) = I_1$

      $S0 = I_1'I_0'T + I_1'I_0T + I_1I_0$

      $= I_1'T(I_0' + I_0) + I_1I_0$

      $= I_1'T + I_1I_0$
    - SBR load signal (**L**)

      $L = I_1'I_0T$

BR Field		Input			MUX 1		Load SBR	
		I1	I0	T	S1	S0	L	
0	0	0	0	0	0	0	0	① CAR + 1
0	0	0	0	1	0	1	0	② JMP
0	1	0	1	0	0	0	0	① CAR + 1
0	1	0	1	1	0	1	1	② CALL
1	0	1	0	x	1	0	0	③ RET
1	1	1	1	x	1	1	0	④ MAP

CALL

In-class assignment

7-13. Suppose that we change the ADD routine listed in Table 7-2 to the following two microinstructions.

```
ADD:    READ    I    CALL    INDR2
        ADD     U    JMP     FETCH
```

What should be subroutine INDR2?

TABLE 7-2 Symbolic Microprogram (Partial)

Label	Microoperations	CD	BR	AD
	ORG 0			
ADD:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ADD	U	JMP	FETCH
	ORG 4			
BRANCH:	NOP	S	JMP	OVER
	NOP	U	JMP	FETCH
OVER:	NOP	I	CALL	INDRCT
	ARTPC	U	JMP	FETCH
	ORG 8			
STORE:	NOP	I	CALL	INDRCT
	ACTDR	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 12			
EXCHANGE:	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	ACTDR, DRTAC	U	JMP	NEXT
	WRITE	U	JMP	FETCH
	ORG 64			
FETCH:	PCTAR	U	JMP	NEXT
	READ, INCPC	U	JMP	NEXT
	DRTAR	U	MAP	
INDRCT:	READ	U	JMP	NEXT
	DRTAR	U	RET	

In-class assignment: Answer

7.13

If I = 0, the operand is read in the first microinstruction and added to AC in the second.

If I = 1, the effective address is read into DR and control goes to INDR2. The subroutine must read the operand into DR.

INDR 2 :	DRTAR	U	JMP	NEXT
	READ	U	RET	---

In-class assignment

7-16. Add the following instructions to the computer of Sec 7-3 (EA is the effective address). Write the symbolic microprogram for each routine as in Table 7-2. (Note that AC must not change in value unless the instruction specifies a change in AC.)

Symbol	Opcode	Symbolic Function	Description
AND	0100	$AC \leftarrow AC \wedge M[EA]$	AND
SUB	0101	$AC \leftarrow AC - M[EA]$	Subtract
ADM	0110	$M[EA] \leftarrow M[EA] + AC$	Add to memory
BTCL	0111	$AC \leftarrow AC \wedge \overline{M[EA]}$	Bit clear
BZ	1000	If $(AC = 0)$ then $(PC \leftarrow EA)$	Branch if AC zero
SEQ	1001	If $(AC = M[EA])$ then $(PC \leftarrow PC + 1)$	Skip if equal
BPNZ	1010	If $(AC > 0)$ then $(PC \leftarrow EA)$	Branch if positive and nonzero

In-class assignment: answer

7.16

	ORG 16			
AND :	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
ANDOP :	AND	U	JMP	FETCH
	ORG 20			
SUB :	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	SUB	U	JMP	FETCH
	ORG 24			
ADM :	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	DRTAC, ACTDR	U	JMP	NEXT
	ADD	U	JMP	EXCHANGE +2

(Table 7.2)

	ORG 28			
BICL :	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	DRTAC, ACTDR	U	JMP	NEXT
	COM	U	JMP	ANDOP
	ORG 32			
BZ :	NOP	Z	JMP	ZERO
	NOP	U	JMP	FETCH
ZERO :	NOP	I	CALL	INDRCT
	ARTPC	U	JMP	FETCH
	ORG 36			
SEQ :	NOP	I	CALL	INDRCT
	READ	U	JMP	NEXT
	DRTAC, ACTDR	U	JMP	NEXT
	XOR (or SUB)	U	JMP	BEQ1
	ORG 69			
BEQ 1 :	DRTAC, ACTDR	Z	JMP	EQUAL
	NOP	U	JMP	FETCH
EQUAL :	INC PC	U	JPM	FETCH
	ORG 40			
BPNZ :	NOP	S	JMP	FETCH
	NOP	Z	JMP	FETCH
	NOP	I	CALL	INDRCT
	ARTPC	U	JMP	FETCH


The End